

Redes Neurais para Processamento de Linguagem Natural

Prof. Denilson Alves Pereira

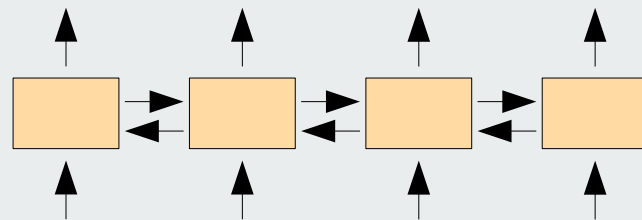
denilsonpereira@ufla.br

<https://sites.google.com/ufla.br/denilsonpereira/>

Universidade Federal de Lavras

Instituto de Ciência Exatas e Tecnológicas

Departamento de Ciência da Computação



Conteúdo



Introdução ao Processamento de Linguagem Natural (PLN)

Modelos Sequenciais

Redes Neurais Recorrentes

Modelo *Sequence-to-Sequence*

Mecanismo de Atenção

Transformer

Representação de Palavras

Modelo de Linguagem

Transferência de Aprendizagem

Representação de Documentos

Aplicações de PLN



Introdução ao Processamento de Linguagem Natural

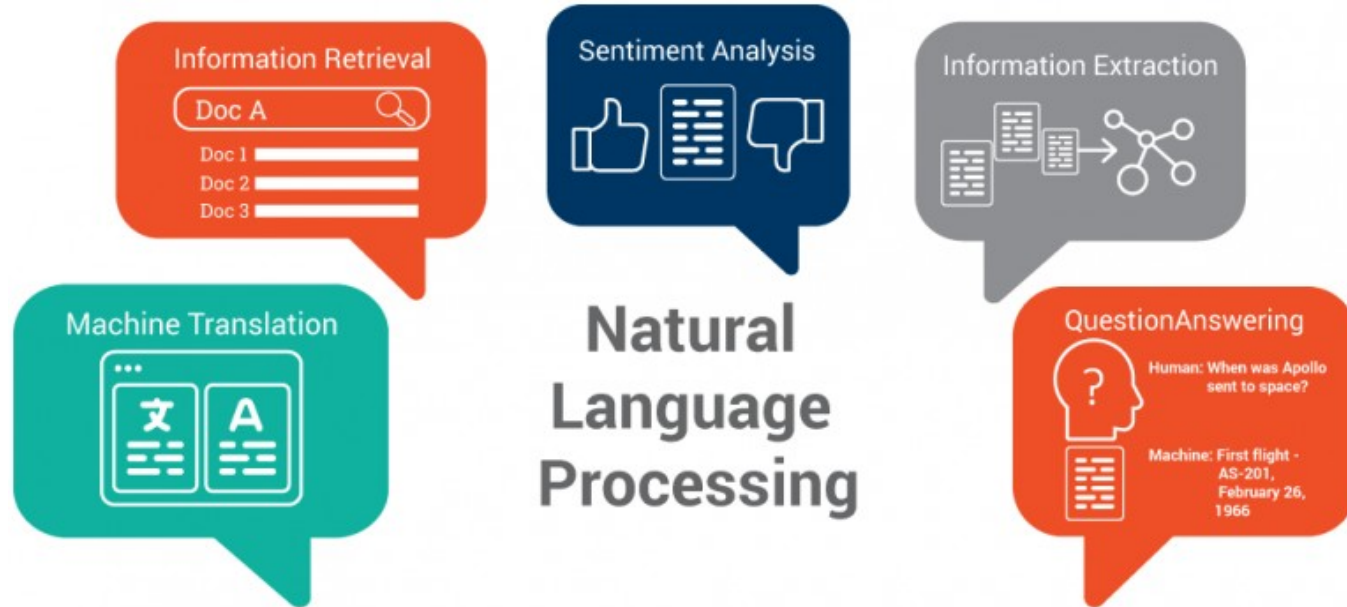
Introdução Proc. de Linguagem Natural



Processamento de Linguagem Natural (PLN) é um campo da Linguística e da Aprendizagem de Máquina focado no entendimento da linguagem humana

Tem como objetivo entender o contexto das palavras

Introdução Proc. de Linguagem Natural



Referência.: <https://www.ontotext.com/blog/top-5-semantic-technology-trends-2017/>
<https://neptune.ai/blog/recurrent-neural-network-guide>

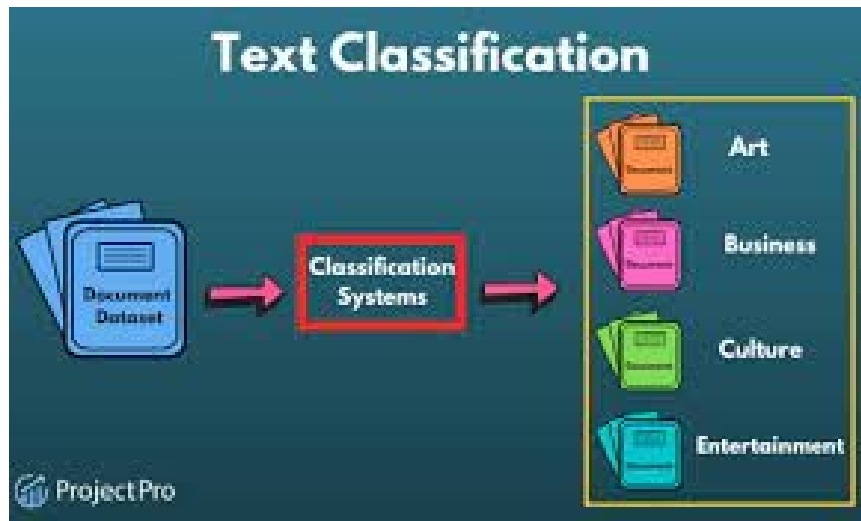
Exemplos de Tarefas de PLN

Classificação de texto

Analisar sentimentos

Detectar spam

Identificar se duas
sentenças estão
relacionadas



Referência.:

<https://www.projectpro.io/article/machine-learning-nlp-text-classification-algorithms-and-models/523>

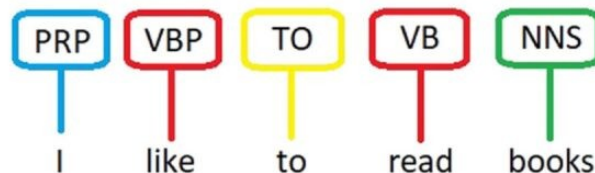
Exemplos de Tarefas de PLN

Classificação de palavras

Identificar a classe gramatical
(substantivo, verbo, adjetivo)

Reconhecer entidades (local,
pessoa, organização)

POS Tagging



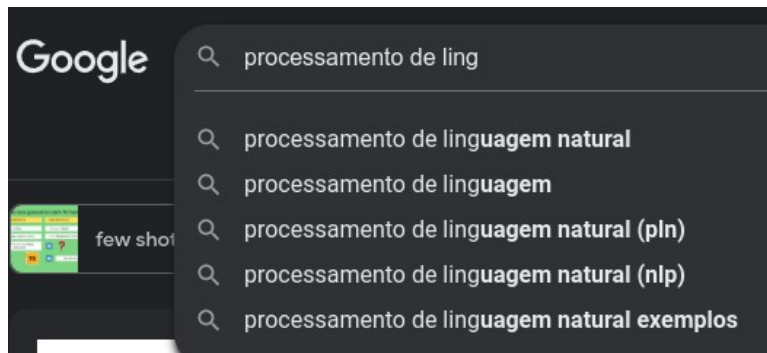
Referência.: <https://byteiota.com/pos-tagging/>

Exemplos de Tarefas de PLN

Geração de conteúdo textual

Completar um *prompt*: próxima(s) palavra(s)

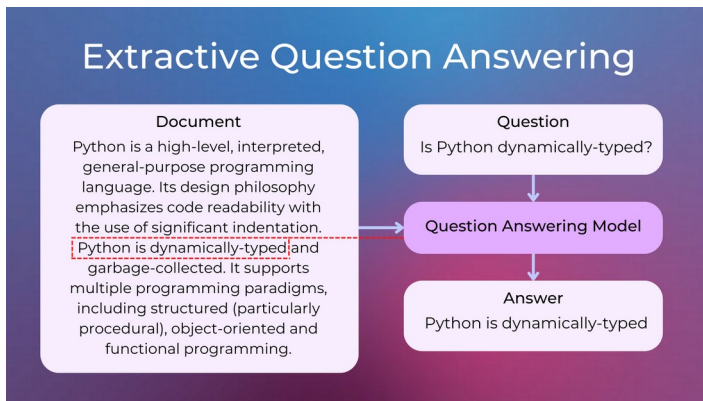
Preencher um espaço em branco em um texto com palavras mascaradas



Exemplos de Tarefas de PLN

Extração de respostas de um texto

extrair a resposta para uma pergunta dado um contexto



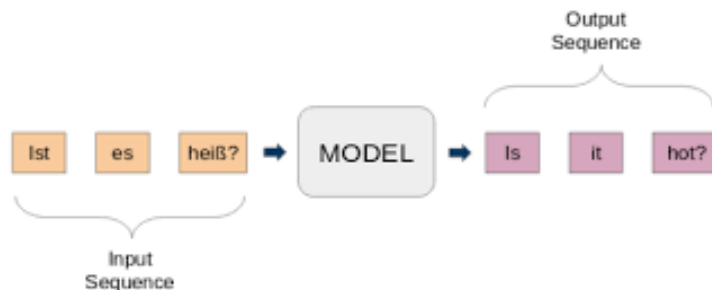
Referência.: <https://www.nlplanet.org/course-practical-nlp/02-practical-nlp-first-tasks/17-question-answering.html>

Exemplos de Tarefas de PLN

Geração de nova sentença a partir de uma entrada

Traduzir um texto para um novo idioma

Sumarizar um texto



Referência.: <https://www.analyticsvidhya.com/blog/2019/01/neural-machine-translation-keras/>

Introdução Proc. de Linguagem Natural

PLN não está limitado à linguagem escrita

Tarefas mais complexas:

Transcrição de áudio

Descrição de uma imagem



Referência.: <https://www.ics-dubai.com/blogs/what-is-audio-transcription>



Modelos Sequenciais

Modelos Sequenciais



Modelos matemáticos que tratam de dados que ocorrem em sequência

Propriedades de dados sequenciais:

- Elementos na sequência podem se repetir

- Seguem uma ordem (arranjo contextual)

- Tem um comprimento variável

Exemplos de Dados Sequenciais



Textos

Áudios (ex: fala e música)

Imagens em movimento ou vídeos

Séries temporais (ex: mercado de ações)

Sequências de DNA, estruturas de proteínas

Modelos Sequenciais



Redes neurais *feed-forward* (densas) não são capazes de modelar dados sequenciais

Funcionam bem com dados de entrada de tamanho fixo

Não consideram a estrutura dos dados

Modelos Sequenciais



Redes neurais convolucionais são limitadas para modelar dados sequenciais

Foram projetadas para processar estruturas, ou *grids* de dados, tal com uma imagem

Não podem ordenar a sequência corretamente

Como Modelar Dados Sequenciais

	Supervised learning	Sequence modelling
Data	$\{x, y\}_i$	$\{x\}_i$
Model	$y \approx f_{\theta}(x)$	$p(x) \approx f_{\theta}(x)$
Loss	$\mathcal{L}(\theta) = \sum_{i=1}^N l(f_{\theta}(x_i), y_i)$	$\mathcal{L}(\theta) = \sum_{i=1}^N \log p(f_{\theta}(x_i))$
Optimisation	$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$	$\theta^* = \arg \max_{\theta} \mathcal{L}(\theta)$

O modelo estima a probabilidade da sequência

Como Modelar Dados Sequenciais



Considere a sentença:

Santos Dumont inventou o _____

Predizer a próxima palavra pode ser modelado como a probabilidade condicional:

$$p(x|\text{contexto})$$

Probabilidade Condicional



Santos Dumont inventou o _____

$$p(x_t) = p(x_t|x_1, x_2, \dots, x_{t-1})$$

Alvo	$p(x \text{contexto})$
avião	0.9
navio	0.0001
telefone	0.00003

Probabilidade Conjunta e Condicional



Para prever uma sequência de palavras, pode-se calcular a probabilidade conjunta das condicionais

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1})$$

Probabilidade Conjunta e Condicional



Alvo	$p(x \text{contexto})$
Santos	$p(x_1)$
Santos Dumont	$p(x_2 x_1)$
Santos Dumont inventou	$p(x_3 x_2, x_1)$
Santos Dumont inventou o	$p(x_4 x_3, x_2, x_1)$
Santos Dumont inventou o avião	$p(x_5 x_4, x_3, x_2, x_1)$

N-Gramas



Problema: não escalável para sentenças muito longas ou parágrafos

Solução: n-gramas

n é o número de palavras para a probabilidade conjunta e condicional

Contexto é perdido para sentenças maiores do que n

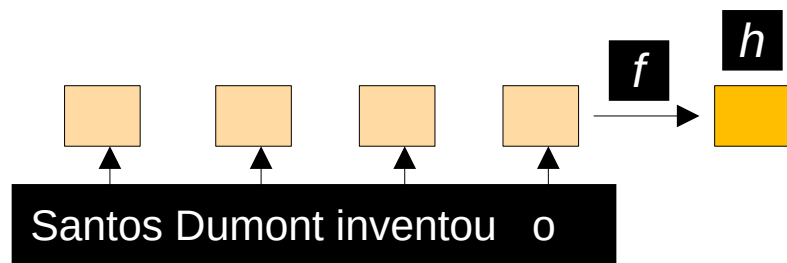
Prob. Conjunta Condicional n-gramas



Alvo	$p(x \text{contexto})$ e $n = 2$
Santos	$p(x_1)$
Santos Dumont	$p(x_2 x_1)$
Santos Dumont inventou	$p(x_3 x_2, x_1)$
Santos Dumont inventou o	$p(x_4 x_3, x_2)$
Santos Dumont inventou o avião	$p(x_5 x_4, x_3)$

Vetorização do Contexto

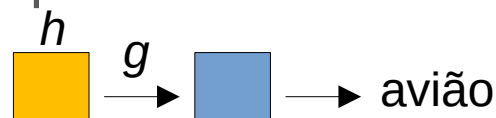
Sumariza a sequência de entrada em um vetor de forma que esse vetor é usado pra prever a próxima palavra



A função f sumariza o contexto h tal que

$$p(x_t) = p(x_t | x_1, x_2, \dots, x_{t-1}) \approx p(x_t | h)$$

Uma função g aplicada ao contexto h produz uma distribuição de probabilidade



Vetorização do Contexto



Vantagens:

- A ordem é preservada

- Pode operar em sequências de tamanho variável

- Pode aprender e, daí, ser diferenciável (*backpropagation*)

- Contexto é preservado em sequências curtas

Vamos aprender como as redes neurais usam a vetorização do contexto para prever a próxima palavra



Redes Neurais Recorrentes

Redes Neurais Recorrentes (RNNs)



Recurrent Neural Network (RNN)

Componentes principais:

Entrada

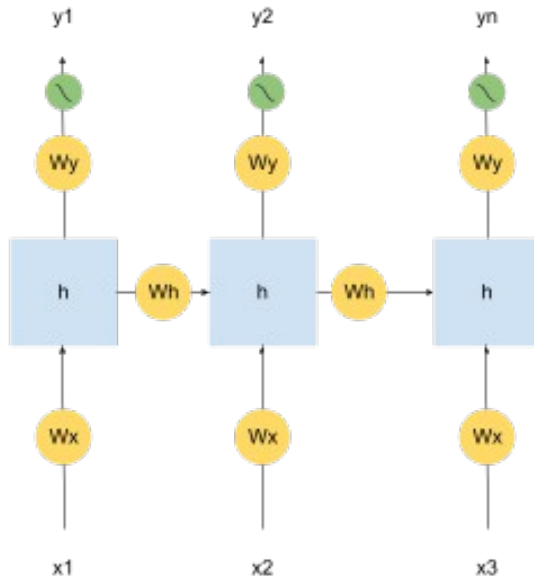
Estado oculto (*hidden state*)

Pesos (parâmetros)

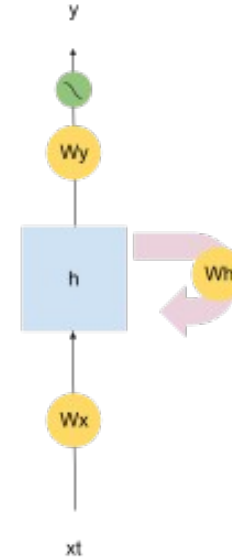
Saída

O estado oculto faz o papel da vetorização do contexto

Redes Neurais Recorrentes (RNNs)

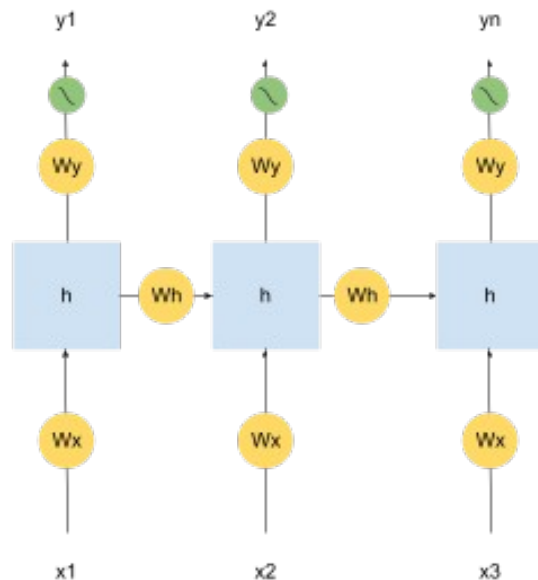


Representação aberta



Representação fechada

Redes Neurais Recorrentes (RNNs)



x_t : entrada no tempo t

h_t : vetor de contexto no tempo t (memória)

W_h : pesos para os estados ocultos

W_x : pesos para as entradas

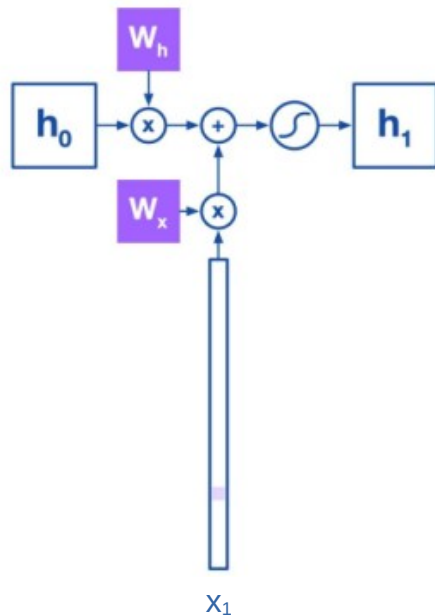
W_y : pesos para as saídas

O estado oculto h é denotado por:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

Cada conjunto de pesos é igual para todas as camadas h (pesos compartilhados)

Redes Neurais Recorrentes (RNNs)



x_t : entrada no tempo t

h_t : vetor de contexto no tempo t (memória)

W_h : pesos para os estados ocultos

W_x : pesos para as entradas

W_y : pesos para as saídas

O estado oculto h é denotado por:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

h_0 é inicializado com zero

Redes Neurais Recorrentes (RNNs)



RNN prediz a saída do último estado oculto

A predição é uma tarefa de classificação

A função Softmax é usada para assegurar a probabilidade sobre todas as possíveis palavras na sentença

Tipos de Redes Neurais Recorrentes



RNN Bidirecional

LSTM (*Long Short Term Memory*)

GRU (*Gated Recurrent Unit*)

Rede Neural Recorrente Bidirecional



A RNN unidirecional modela a próxima saída dada a sequência anterior

Entretanto, o contexto pode depender da sequência futura:

Eu estou faminto, vou comer a metade do bolo.

Eu estou pouco faminto, vou comer só uma fatia.

Eu estou muito faminto, posso comer um bolo inteiro.

Rede Neural Recorrente Bidirecional



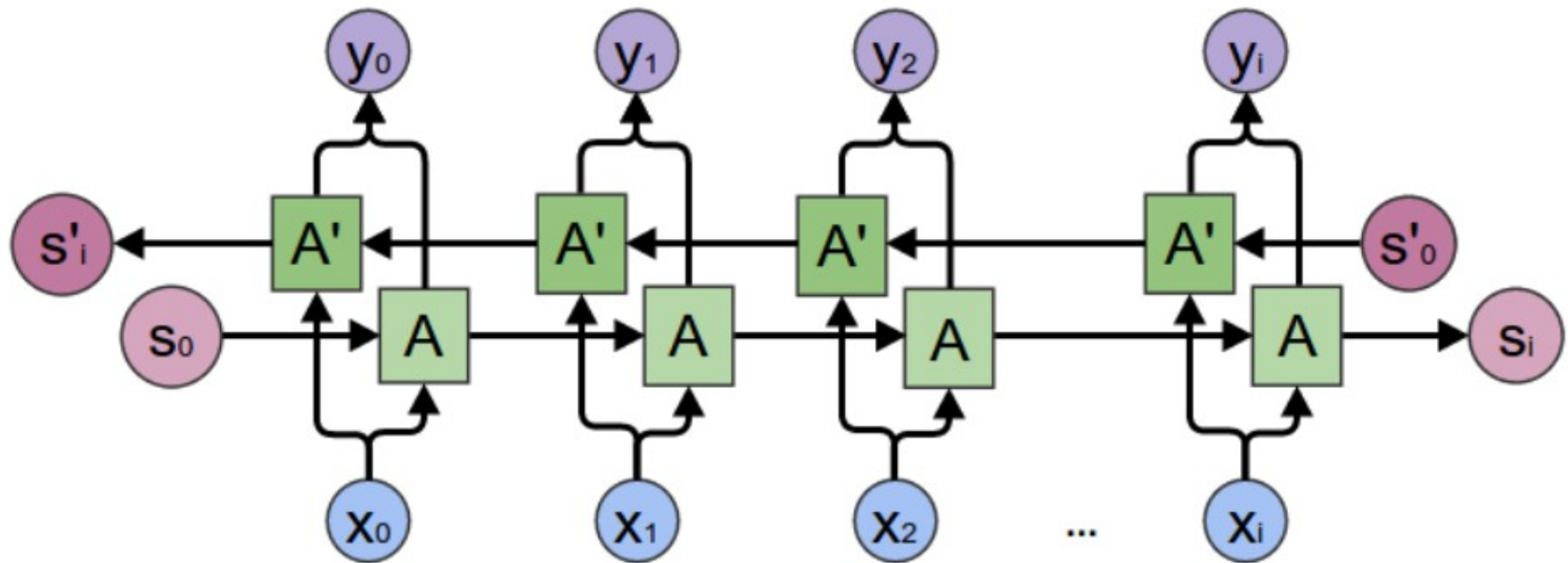
A RNN bidirecional modela a próxima saída com base na sequência passada e na sequência futura

Duas RNNs independentes trabalham juntas

Uma funciona na direção pra frente

A outra funciona na direção pra trás

Rede Neural Recorrente Bidirecional



Referência.: <https://neptune.ai/blog/recurrent-neural-network-guide>

LSTM (*Long Short Term Memory*)



A LSTM visa suprir a deficiência de memória de longo prazo da RNN

Visa preservar informações contextuais distantes em uma sequência



João, que é professor da UFLA, está na UFMG

LSTM (*Long Short Term Memory*)

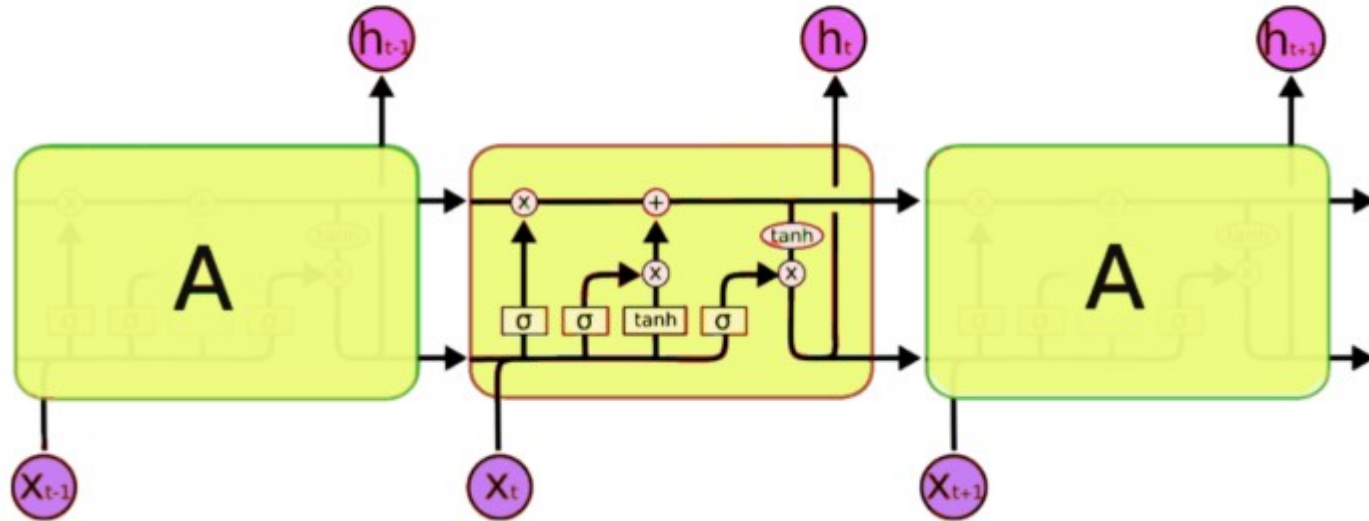


Matematicamente, visa garantir que o gradiente flua por um longo período de tempo e não desapareça (*vanish*) ou exploda (*explode*)

Possui quatro redes neurais interagindo de forma a preservar e compartilhar longas informações contextuais

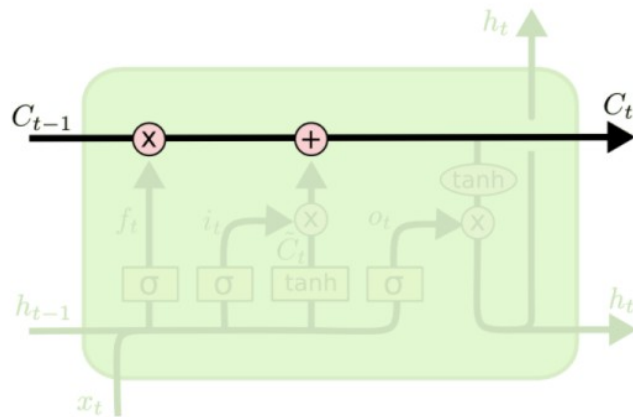
RNN possui apenas uma única camada de rede neural

LSTM (*Long Short Term Memory*)



Referência.: <https://neptune.ai/blog/recurrent-neural-network-guide>

LSTM (*Long Short Term Memory*)

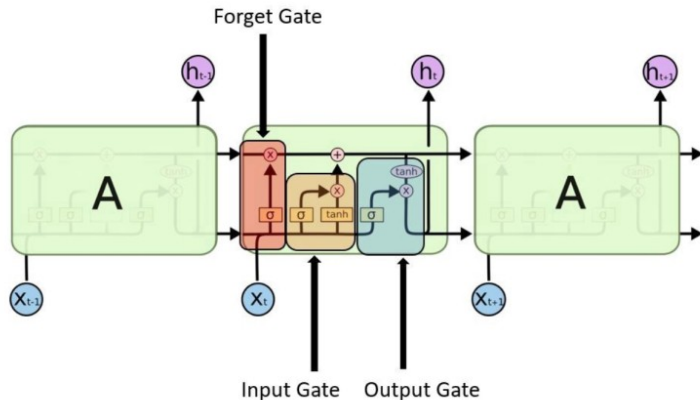


Cell State (C_t): memória de longo prazo

Permite que informações anteriores sejam armazenadas na rede LSTM

É modulada pelas portas internas, chamadas: porta de esquecimento (***forget gate***) e porta de entrada (***input gate***)

LSTM (*Long Short Term Memory*)

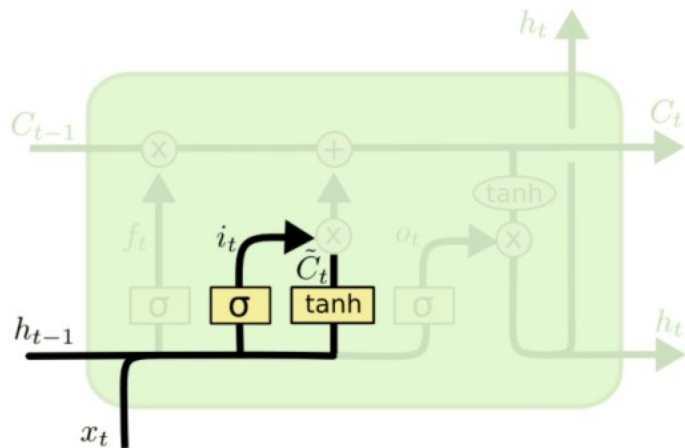


Forget Gate: indica quais informações devem ser preservadas e quais devem ser esquecidas

Verifica o estado inicial, oculto, (*hidden state*) h_{t-1} e a entrada x_t , e retorna como saída, por meio da função *sigmoid*, um número entre 0 e 1 para cada *cell state* C_{t-1}

Retorno igual a 1 significa armazene a informação, e 0, exclua-a

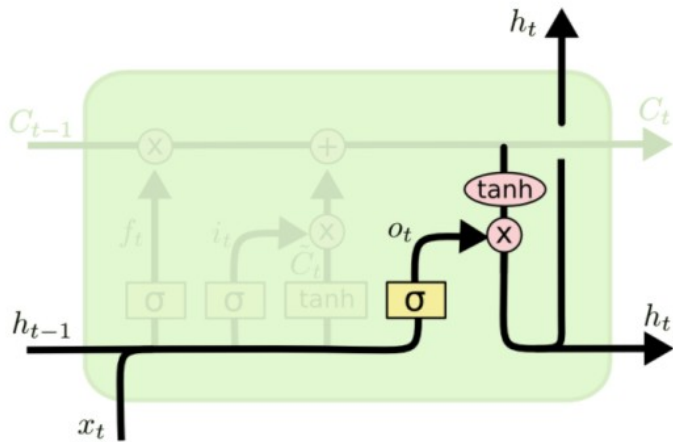
LSTM (*Long Short Term Memory*)



Input Gate: toma o estado inicial h_{t-1} e a entrada x_t , e os passa pelas funções não lineares *sigmoid* (retorna valores entre 0 e 1) e *tanh* (retorna valores entre -1 e 1)

Isso ajuda a *cell state* a salvar novas informações e simultaneamente atualizar a *cell state* de C_{t-1} para C_t , após passar pelo *forget gate*

LSTM (Long Short Term Memory)



Output Gate: saída baseada no *cell state* C_t , no estado inicial h_{t-1} e na entrada x_t .

h_{t-1} e x_t passam pela *sigmoid* e produzem valores entre 0 e 1. Simultaneamente, C_t passa pela *tanh* e produz valores entre -1 e 1.

Os dois resultados são multiplicados e produzem a saída, gerando o que é relevante para o contexto.

Referência.: <https://neptune.ai/blog/recurrent-neural-network-guide>

GRU (*Gated Recurrent Unit*)



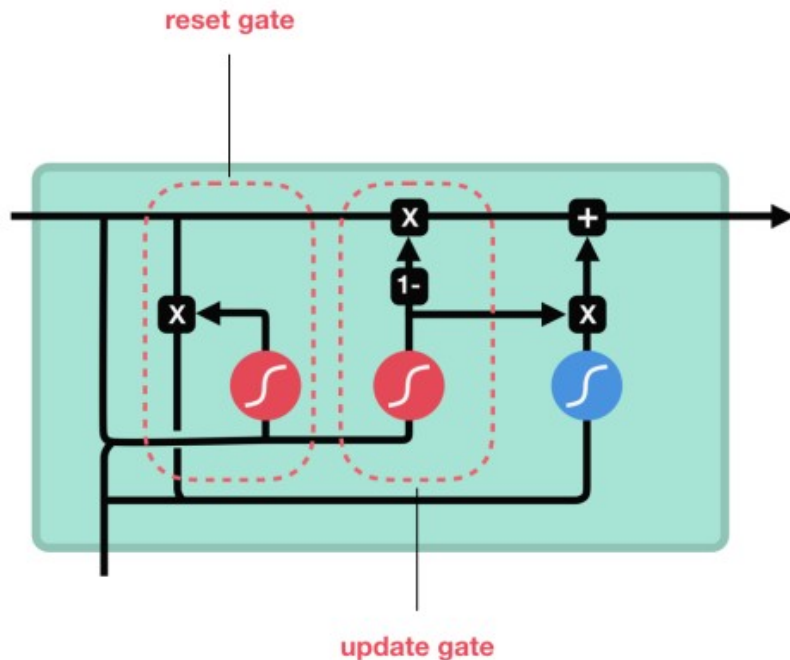
GRU é a geração mais nova das redes neurais recorrentes

É uma versão modificada da LSTM, mais simples e mais rápida

Usa o estado oculto (*hidden state*) para transferir informação, ao invés do *cell state*

Tem apenas duas portas: *reset gate* e *update gate*

GRU (*Gated Recurrent Unit*)



Reset Gate: usado para decidir quanto da informação passada deve ser esquecida

Update Gate: seleciona informação que necessita ser adicionada e/ou ignorada. Similar ao input gate da LSTM



Modelo

Sequence-to-Sequence

Modelo *Sequence-to-Sequence*



Objetivo:

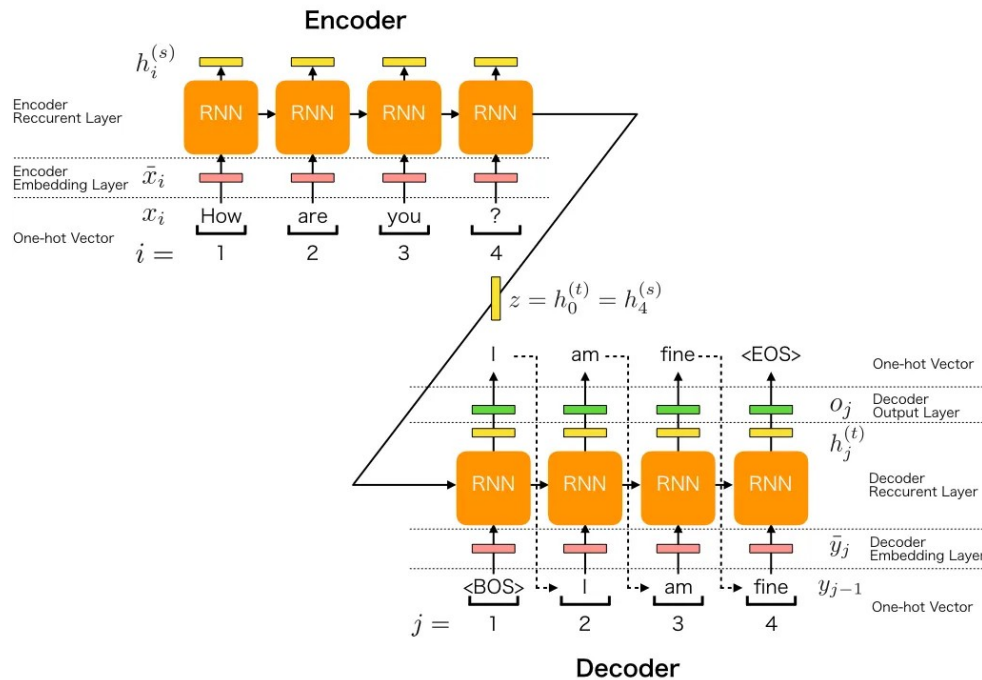
Mapear um vetor de entrada de tamanho fixo para um vetor de saída, também de tamanho fixo

Tamanhos da entrada e da saída podem ser diferentes

Consiste de duas partes: codificador (*encoder*) e decodificador (*decoder*)

Cada parte é uma camada RNN, LSTM ou *Transformer*

Modelo *Sequence-to-Sequence*



Encoder-Decoder



Encoder

Processa a sequência de entrada e comprime a informação em um vetor de contexto h_t de tamanho fixo

Decoder

É inicializado com o vetor de contexto h_t e emite a saída transformada

Encoder-Decoder



Da perspectiva probabilística:

Método geral para aprender a distribuição condicional sobre uma sequência de tamanho variável condicionada a outra sequência de tamanho variável

$$p(y_1, y_2, \dots, y_{T'} \mid x_1, x_2, \dots, x_T)$$

T' e T podem ter tamanhos diferentes

Encoder-Decoder



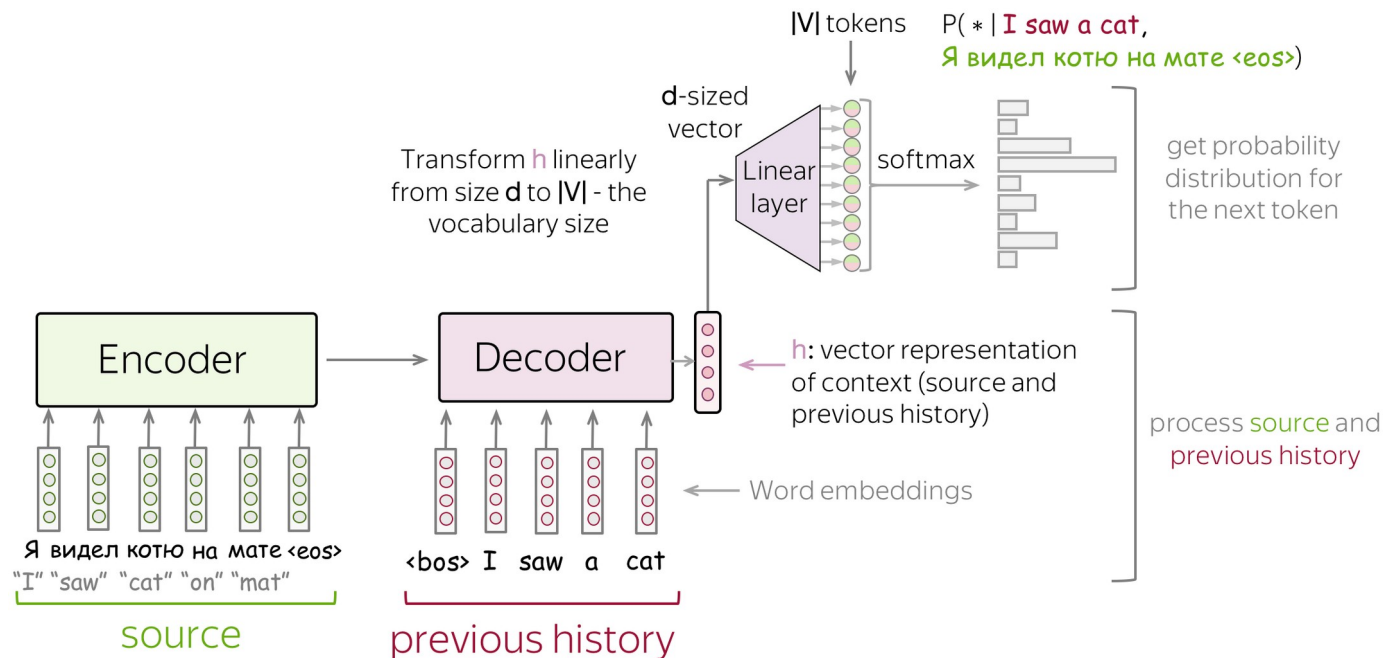
Language Models:
$$P(y_1, y_2, \dots, y_n) = \prod_{t=1}^n p(y_t | y_{<t})$$

Conditional

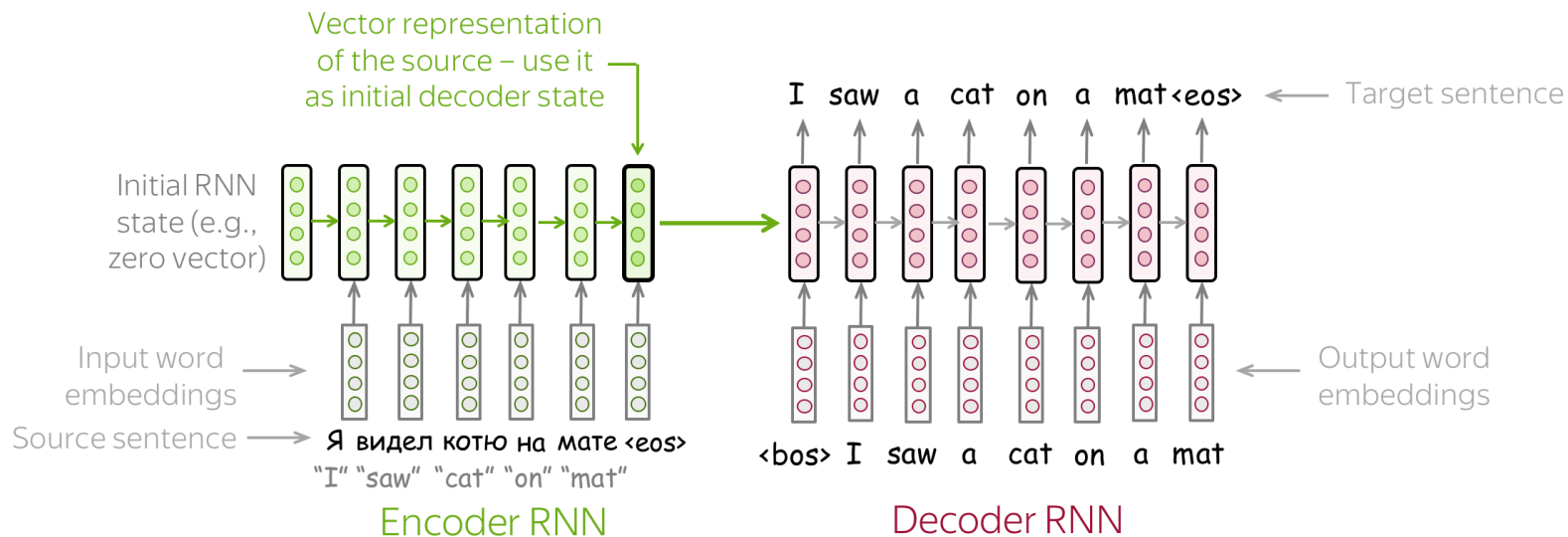
Language Models:
$$P(y_1, y_2, \dots, y_n, |x) = \prod_{t=1}^n p(y_t | y_{<t}, x)$$

condition on source x

Encoder-Decoder



Encoder-Decoder



Encoder-Decoder



Problema:

Não funciona bem para sequências longas

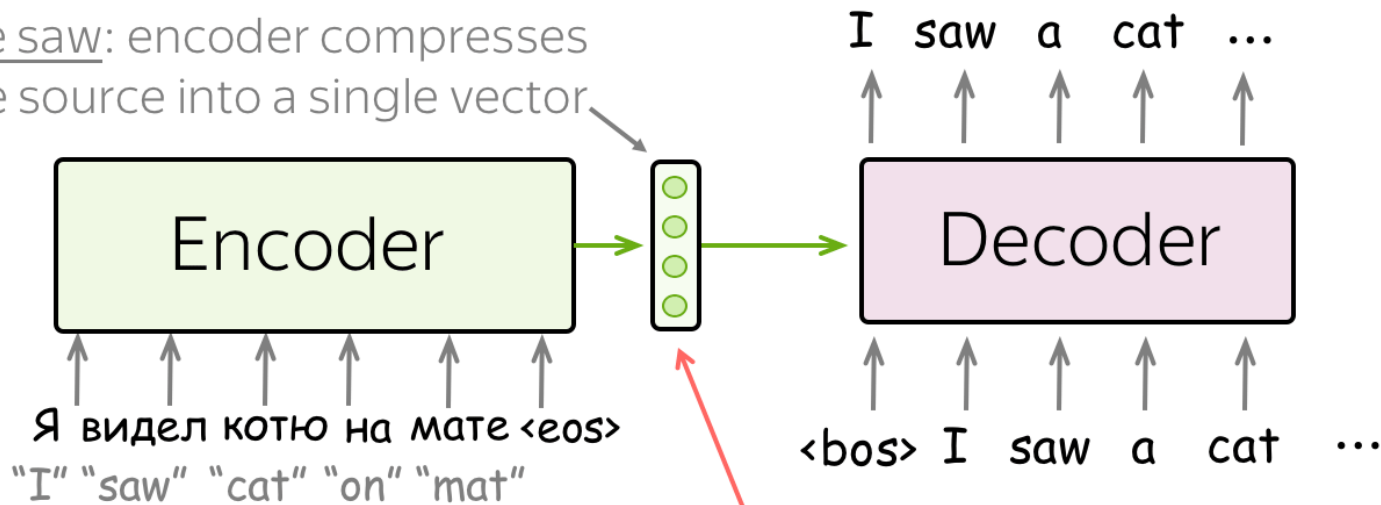
Vetor de contexto tem de codificar muita informação em um tamanho fixo

Solução:

Adicionar um Mecanismo de Atenção

Encoder-Decoder

We saw: encoder compresses the source into a single vector



Problem: this is a bottleneck!

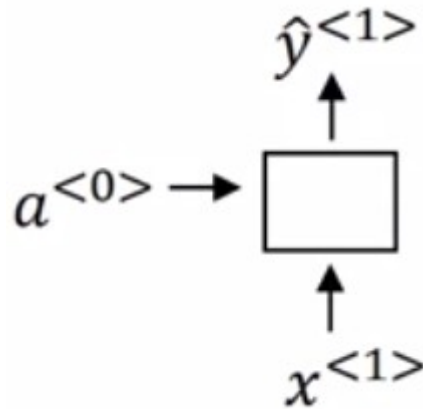


Diferentes Arquiteturas de RNNs

Exemplos de Dados Sequenciais

Speech recognition		→	"The quick brown fox jumped over the lazy dog."
Music generation	$\emptyset$	→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAACTAG	→	AGCCCCTGTGAGGAACTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger .

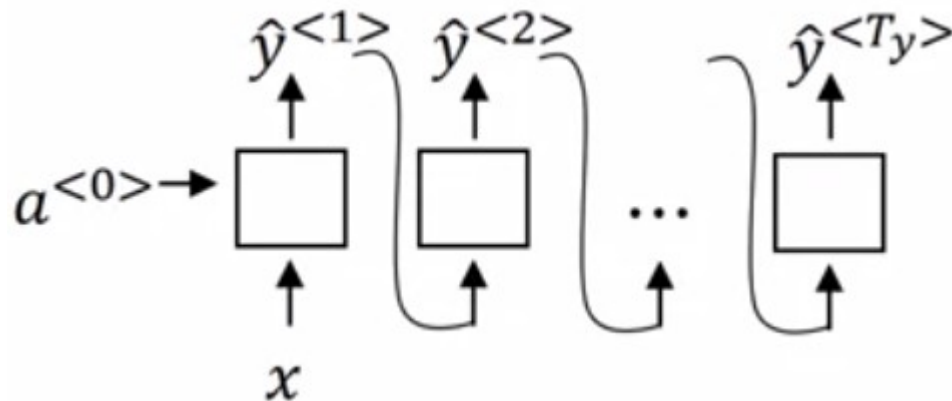
Arquitetura um-pra-um



Uma entrada X
Uma saída Y

Arquitetura padrão,
genérica, sem aplicação
em RNNs

Arquitetura um-pra-muitos

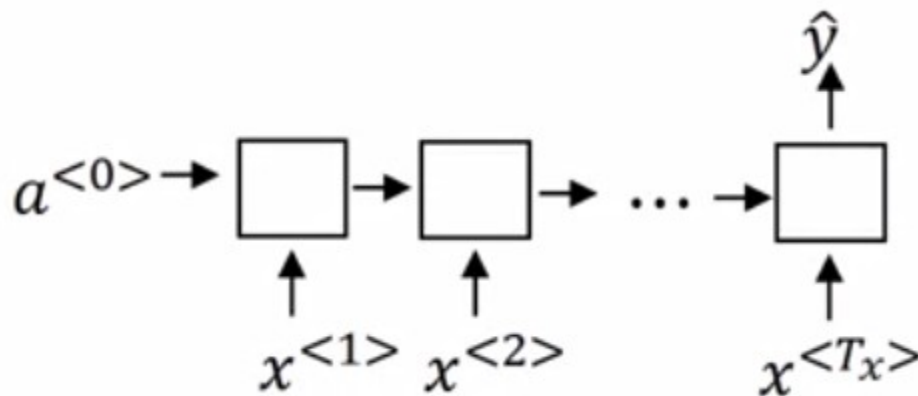


Uma entrada X
Muitas saídas Y

Ex: geração de música

Dada uma nota musical inicial (ou nenhuma), gera-se uma sequência de notas musicais

Arquitetura muitos-pra-um



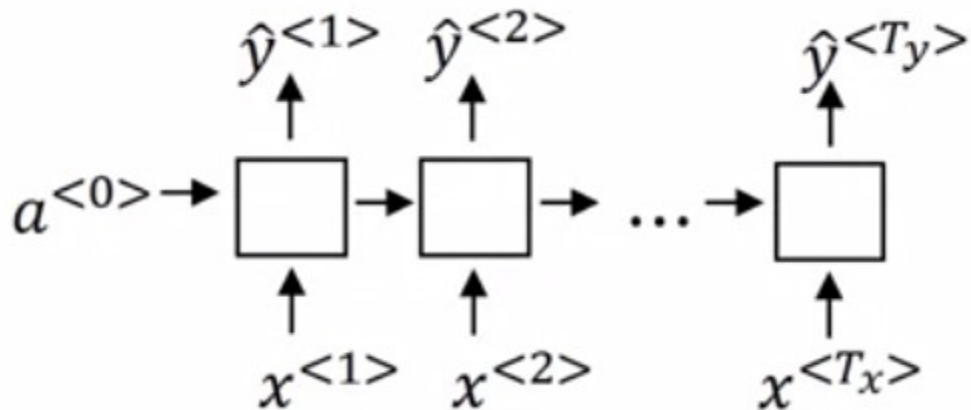
Muitas entradas X

Uma saída Y

Ex: classificação de sentimentos

Dada uma sequência de palavras descrevendo uma revisão de um produto, retorna-se a classificação em número de estrelas do sentimento

Arquitetura muitos-para-muitos



Muitas entradas X

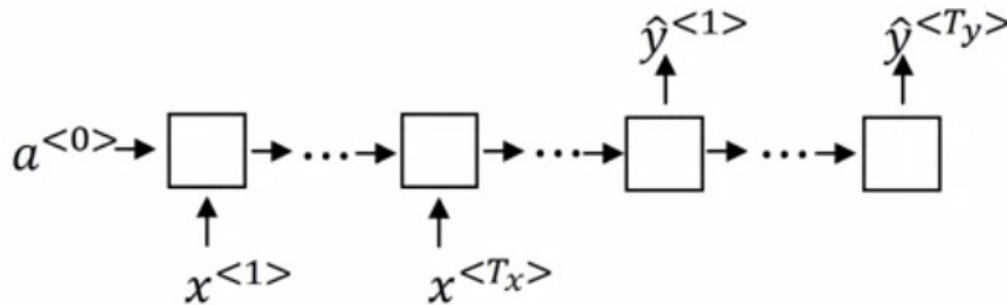
Muitas saídas Y

Tamanho da entrada (T_x)
= tamanho da saída (T_y)

Ex: Reconhecimento de
Entidades Nomeadas

Marca-se as palavras
correspondentes a
nomes de entidades

Arquitetura muitos-para-muitos



Muitas entradas X

Muitas saídas Y

Tamanho da entrada (T_x)
 $\neq$ tamanho da saída (T_y)

Ex: Tradução de texto

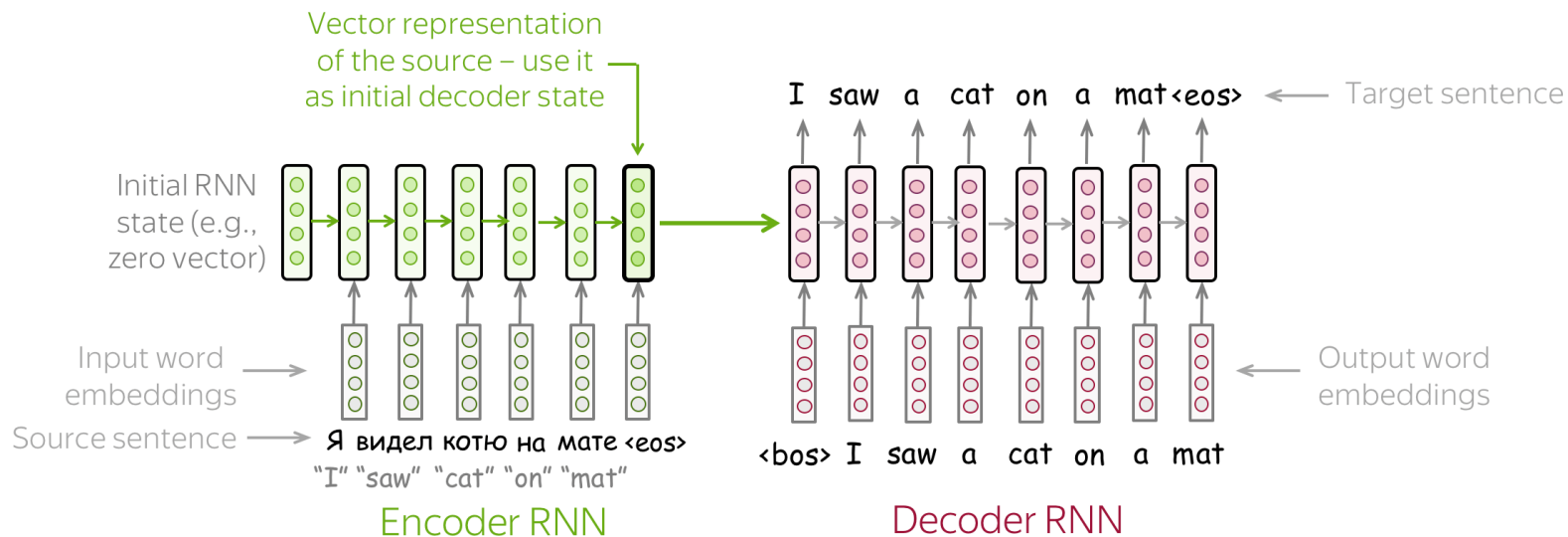
Tradução entre idiomas
não resulta no mesmo
número de palavras

Arquitetura Encoder-
Decoder



Mecanismo de Atenção

Encoder-Decoder SEM Atenção



Mecanismo de Atenção



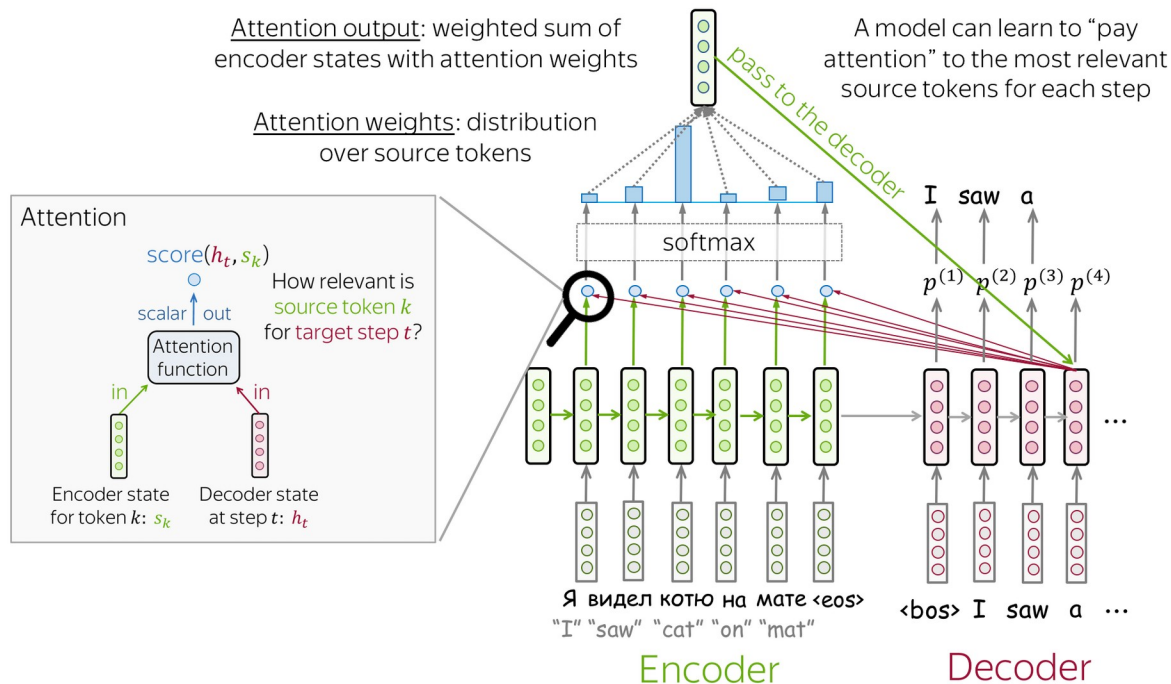
O Mecanismo de Atenção (*Attention*) é parte da rede neural

Encoder fornece representações para todos os *tokens*

Ele não comprime toda a fonte dentro um único vetor

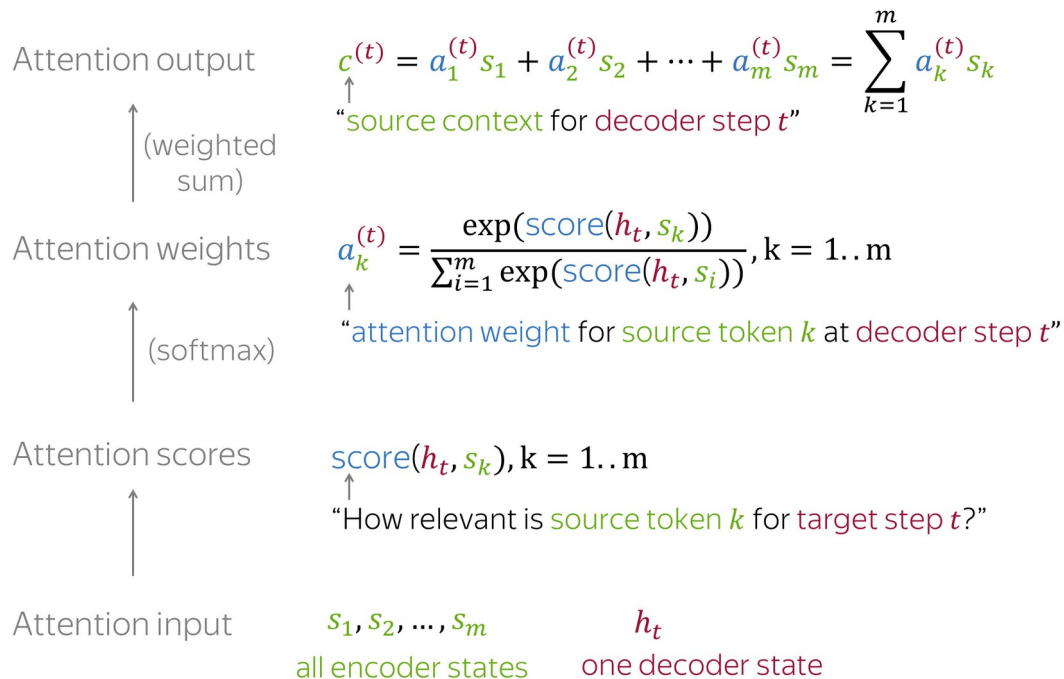
Decoder decide, a cada passo, quais fontes são mais importantes

Mecanismo de Atenção

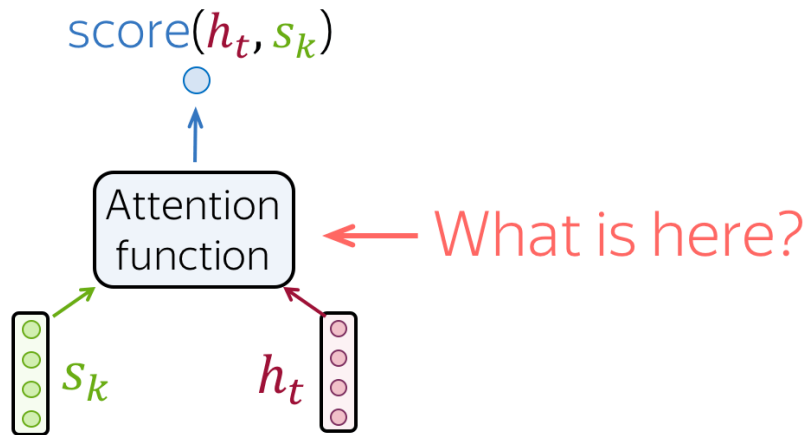


Veja animação em: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Esquema de Computação Genérico



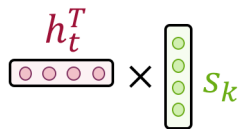
Computando a Pontuação (Score)



Computando a Pontuação (Score)

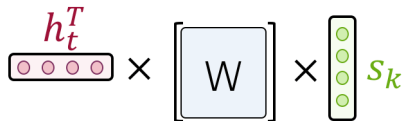
Use uma das três funções mais populares (ou outra qualquer)

Dot-product


$$h_t^T \times s_k$$

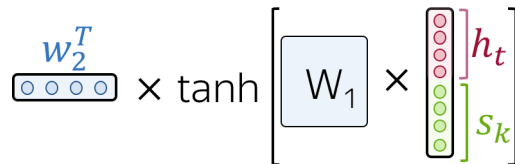
$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear


$$h_t^T \times [W] \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Multi-Layer Perceptron


$$w_2^T \times \tanh \left[W_1 \times \begin{bmatrix} h_t \\ s_k \end{bmatrix} \right]$$

$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$



Transformer

Transformer



Modelo baseado somente em mecanismos de atenção

Sem recorrência ou convolução

Mais rápido para ser treinado

Padrão de fato para tarefas *sequence-to-sequence* e para modelos de linguagem

Mecanismo de *Self-Attention* (Auto-Atenção)



Encoder: a cada passo, os *tokens* “olham” uns para os outros, trocam informação e tentam entender cada um dos outros no contexto de toda a sentença

Decoder: *tokens* do prefixo interagem uns com os outros via mecanismo de *self-attention* e, adicionalmente, “olham” para os estados do *encoder*

Attention X Self-Attention



Attention:

Encoder olha para o seu estado anterior na mesma camada

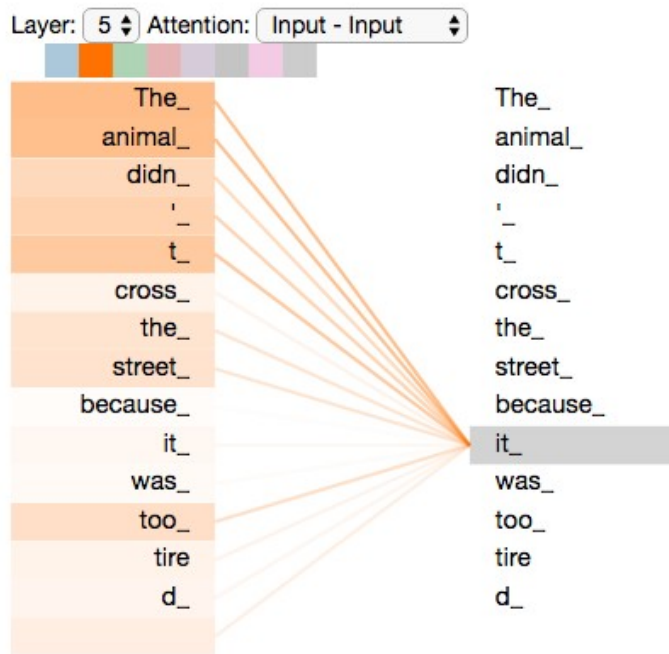
Decoder olha do seu estado corrente para todos os estados do *encoder*

Self-Attention:

Encoder olha para todos os seus estados na mesma camada

Decoder olha para seus estados do prefixo na mesma camada e para todos os estados do *encoder*

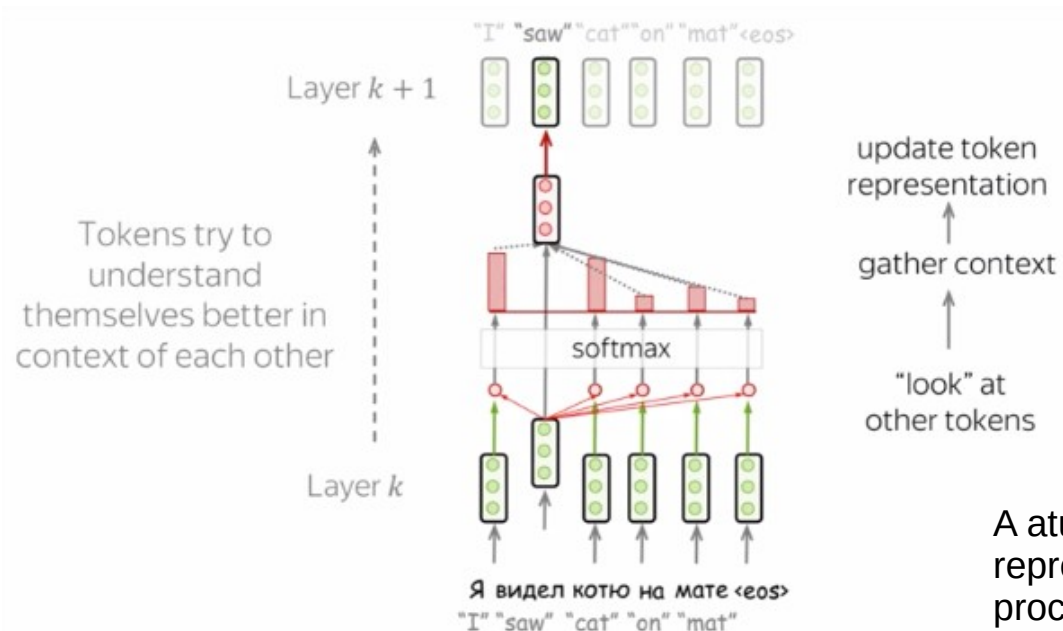
Self-Attention



"The animal didn't cross the street because it was too tired"

"it" está se referindo à animal ou à rua (street)?

Self-Attention



A atualização das representações é processada em paralelo

Query, Key, Value



Cada *token* de entrada recebe 3 representações correspondentes aos papéis que ele desempenha:

Query (pergunta): pede informação

Key (chave): diz que tem alguma informação

Value (valor): fornece a informação

Referência para um exemplo da matemática do Transformer:

<https://levelup.gitconnected.com/understanding-transformers-from-start-to-end-a-step-by-step-math-example-16d4e64e6eb1>

Query, Key, Value

Each vector receives three representations ("roles")

$[W_Q] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{blue} \\ \text{blue} \\ \text{blue} \end{bmatrix}$ **Query:** vector **from** which the attention is looking

"Hey there, do you have this information?"

$[W_K] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{yellow} \\ \text{yellow} \\ \text{yellow} \end{bmatrix}$ **Key:** vector **at** which the query looks to compute weights

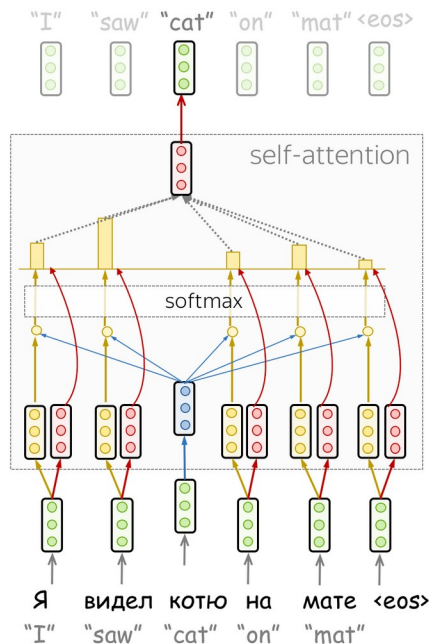
"Hi, I have this information – give me a large weight!"

$[W_V] \times \begin{bmatrix} \text{green} \\ \text{green} \\ \text{green} \end{bmatrix} = \begin{bmatrix} \text{red} \\ \text{red} \\ \text{red} \end{bmatrix}$ **Value:** their weighted sum is attention output

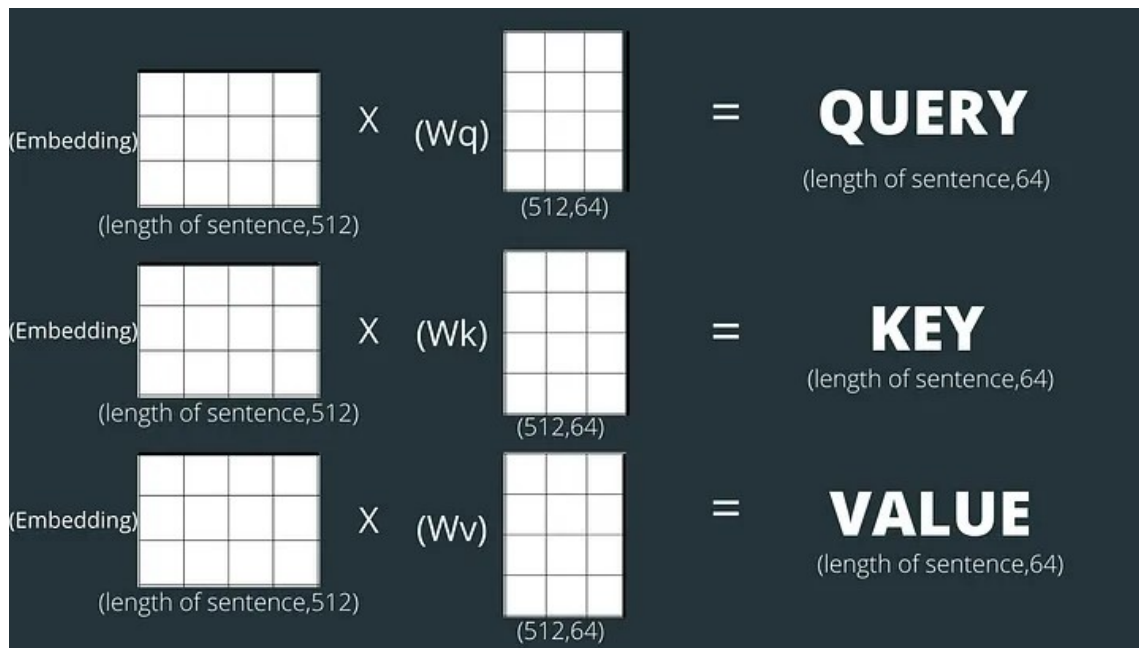
"Here's the information I have!"

$$\text{Attention}(\underset{\substack{\text{from} \\ \text{blue}}}{q}, \underset{\substack{\text{to} \\ \text{yellow}}}{k}, v) = \overbrace{\text{softmax}\left(\frac{qk^T}{\sqrt{d_k}}\right)}^{\text{Attention weights}} v$$

vector dimensionality of K, V



Query, Key, Value



Query, Key, Value

Inputs	je	suis	étudiant
Embedding	x1	x2	x3
Query	q1	q2	q3
Key	k1	k2	k3
Value	v1	v2	v3
Score	$q1*k1=146$	$q1*k2=87$	$q1*k3=24$
Divide by 8	18.25	10.875	3
Softmax	0.82	0.12	0.06
softmax * value	$0.82*v1$	$0.12*v2$	$0.06*v3$
sum	z1	z2	z3

$$\text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \text{3x3 grid} \end{matrix} \times \begin{matrix} \text{K}^T \\ \text{3x3 grid} \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ \text{3x3 grid} \end{matrix}$$

$$= \begin{matrix} \text{Z} \\ \text{3x3 grid} \end{matrix}$$

shape of matrix Z= (length of sentence, dimension of V)

$$z1 = 0.86*v1 + 0.12*v2 + 0.06*v3$$

Masked Self-Attention



No *Decoder*, *self-attention* é diferente do *Encoder*

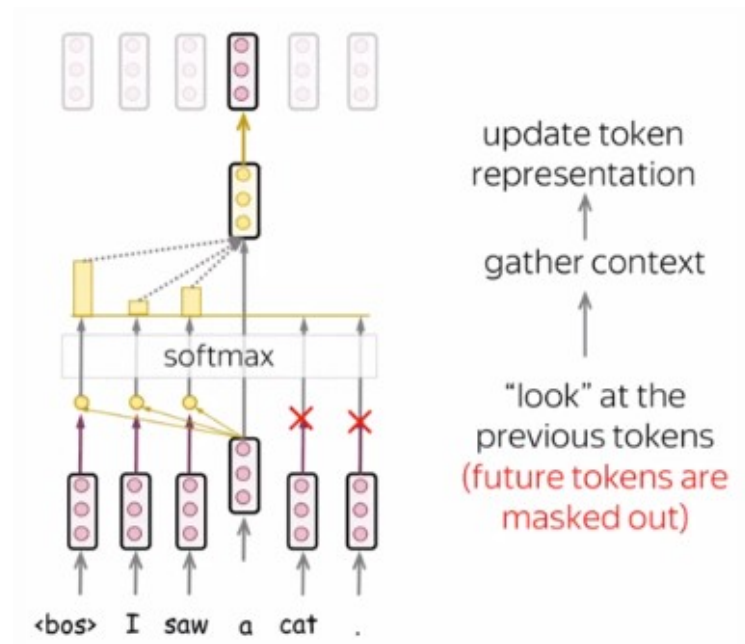
Encoder:

recebe todos os *tokens* de uma vez, e eles podem olhar uns para os outros na sequência de entrada

Decoder:

os *tokens* são gerados um de cada vez, e não se sabe quais *tokens* serão gerados no futuro

Masked Self-Attention



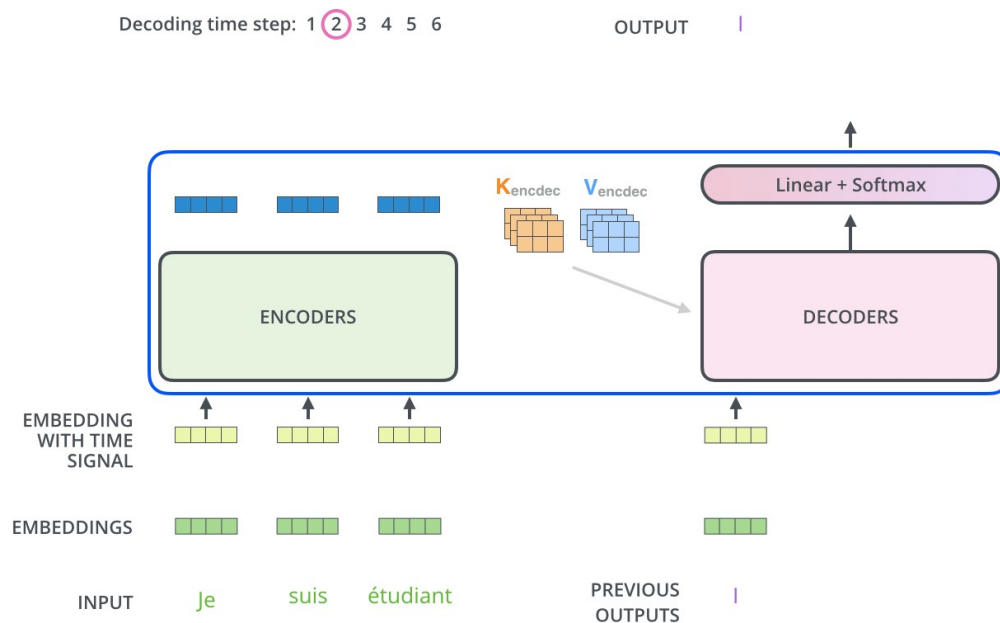
Durante a geração:

- *Tokens* futuros são mascarados
- *Decoder* não olha pra frente

Durante o treinamento:

- *Decoder* não usa máscaras
- *Decoder* “olha” para os *tokens* futuros

Decoder



Veja animação em: <http://jalammar.github.io/illustrated-transformer/>

Eficiência no Treinamento



Modelos recorrentes (RNNs):

$O(\text{len}(\text{fonte}) + \text{len}(\text{alvo}))$ passos por etapa

Transformer:

$O(1)$ constante

Multi-Head Attention

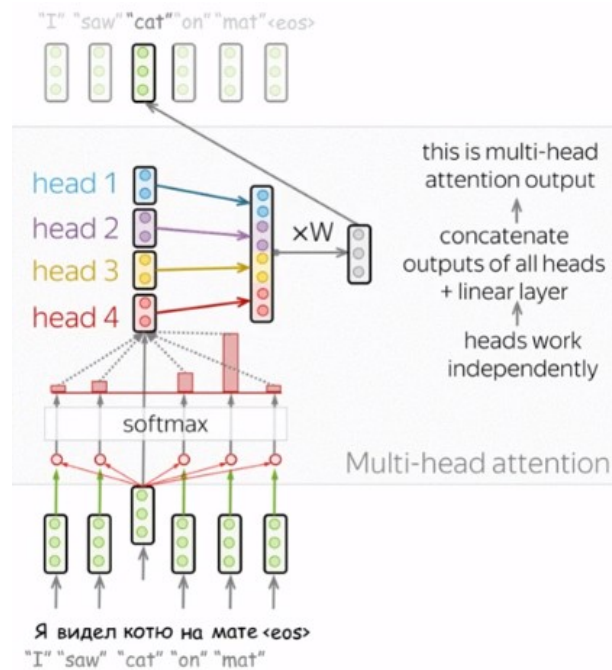
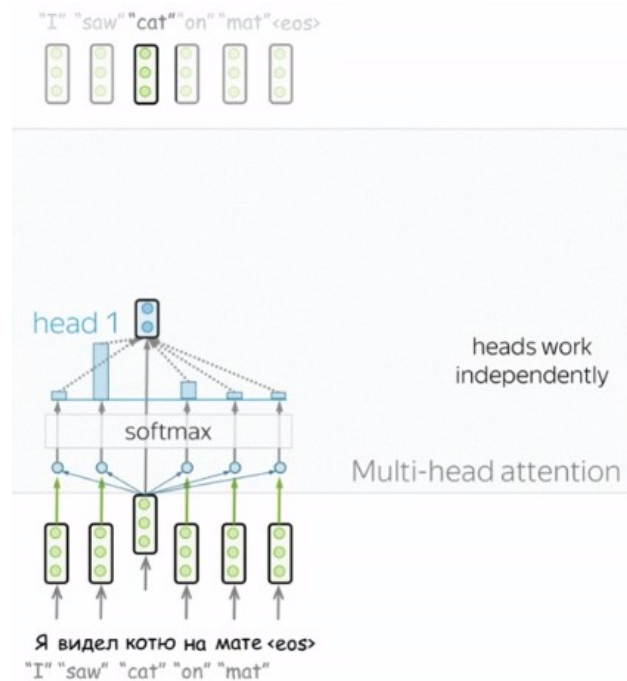


Múltiplas cabeças (*heads*) funcionando de forma independente

Cada cabeça tem um foco diferente
cada palavra é parte de muitas relações
sujeitos definem a inflexão do verbo
verbos definem seus objetos

. . .

Multi-Head Attention

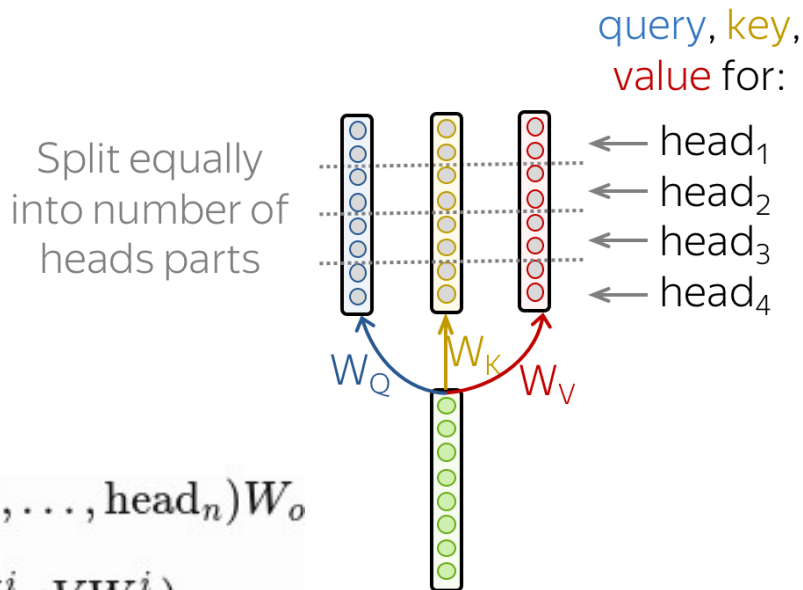


Multi-Head Attention

Implementado como vários mecanismos de atenção, cujos resultados são combinados

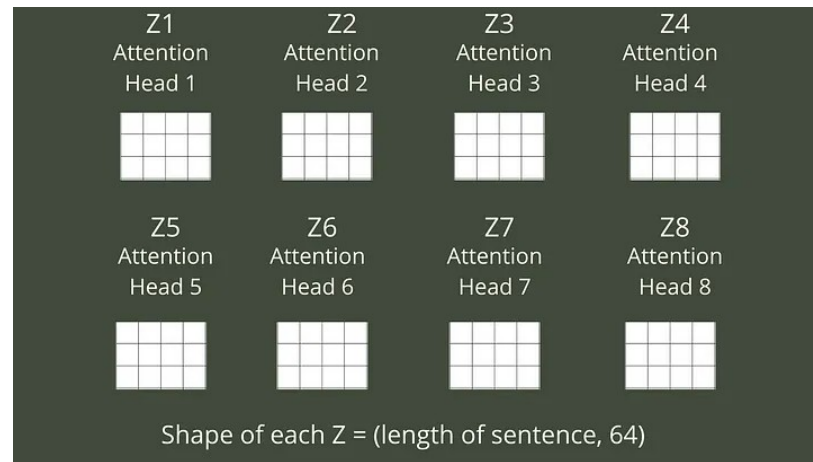
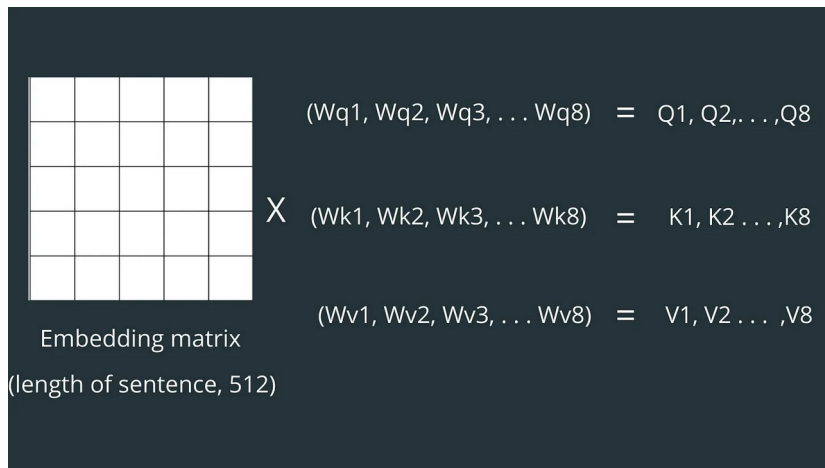
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n)W_o$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$



Multi-Head Attention

Transformer usa multi-head attention com 8 heads



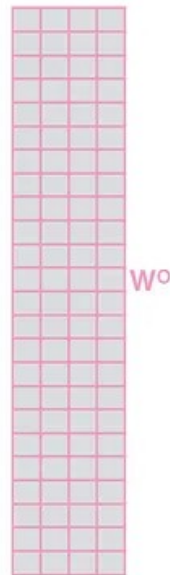
Multi-Head Attention

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

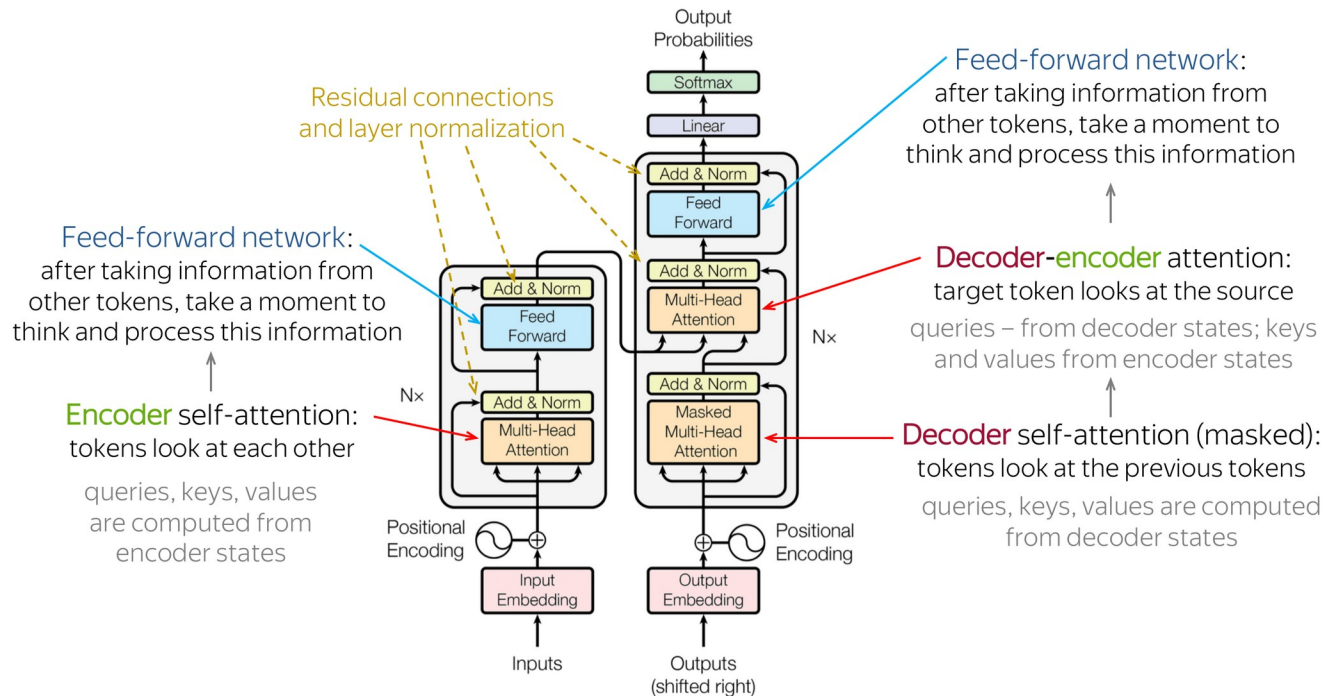
X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Transformer: Arquitetura do Modelo



Transformer: Arquitetura do Modelo



Encoder e *Decoder* funcionam como discutido anteriormente

A arquitetura pode conter várias camadas, usualmente 6

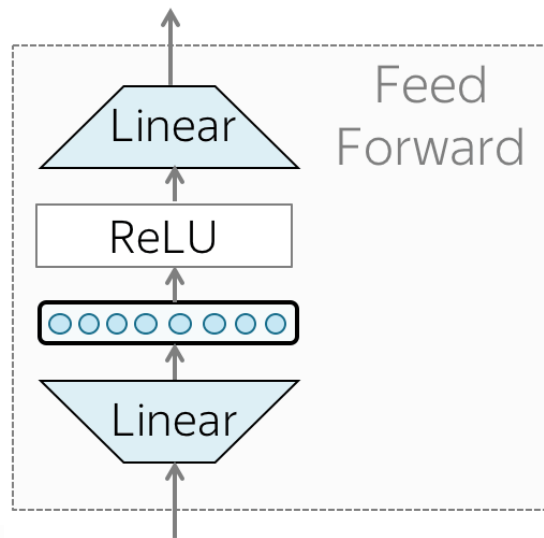
Vamos discutir, a seguir, os outros componentes do modelo

Blocos *Feed-Foward*

Cada camada tem um bloco de rede *feed-foward* com duas camadas lineares e uma função ReLU, de não linearidade, entre elas

Objetivo: processar a nova informação vinda do *Attention*

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



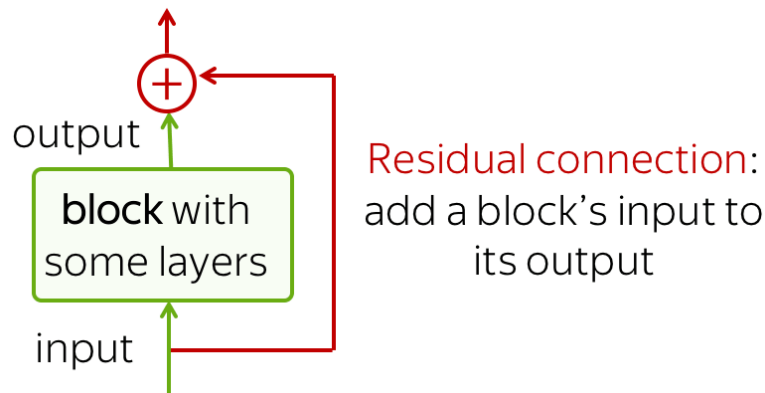
Conexões Residuais

Adiciona a entrada de um bloco a sua saída

Objetivo: facilitar o fluxo do gradiente na rede e permitir empilhar várias camadas

Usado depois de cada bloco Attention e FFN

Parte “Add” em “Add & Norm”



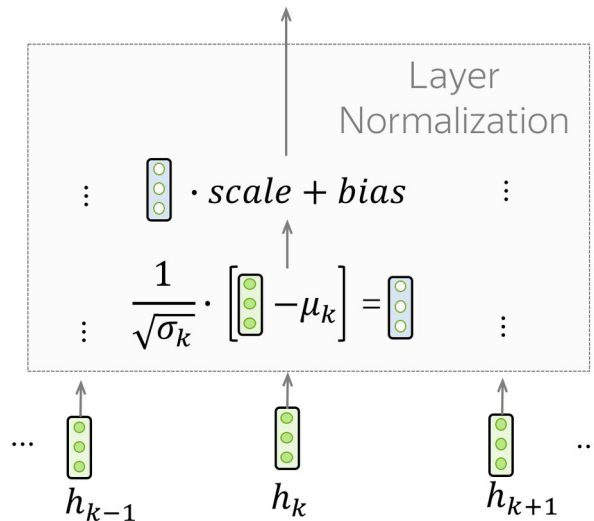
Normalização da Camada

Normaliza o vetor de representação de cada *token*

Possui os parâmetros treináveis *scale* e *bias* para reescalar saídas ou a próxima entrada

Objetivo: controlar o fluxo para a próxima camada, melhorar a estabilidade da convergência e até a qualidade

Parte “Norm” em “Add & Norm”



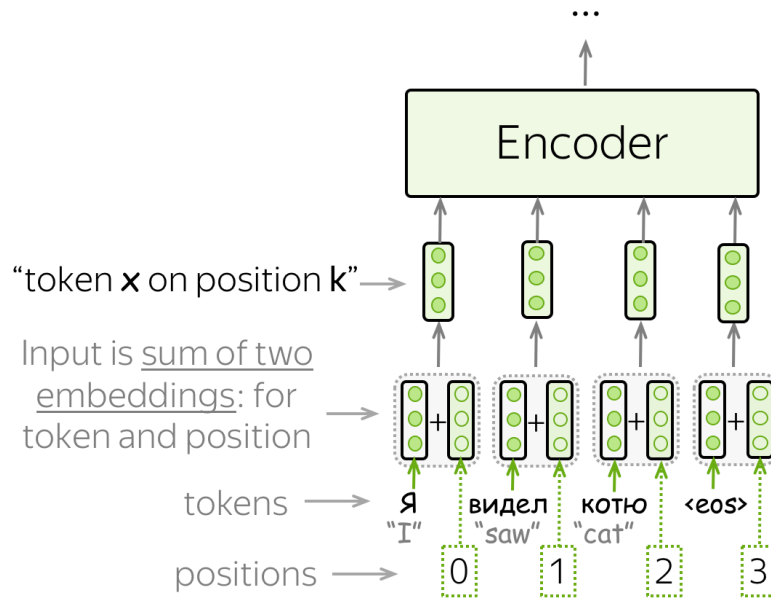
Codificação Posicional

Transformer não conhece a ordem dos tokens de entrada, pois não contém recorrência ou convolução

A posição deve ser informada explicitamente

Dois conjuntos de *embeddings*: um para os *tokens* e outro para as posições

Representação da entrada de um *token*
= soma dos dois *embeddings*



Codificação Posicional

Os *embeddings* posicionais podem ser fixos ou aprendidos

Codificação posicional fixa:

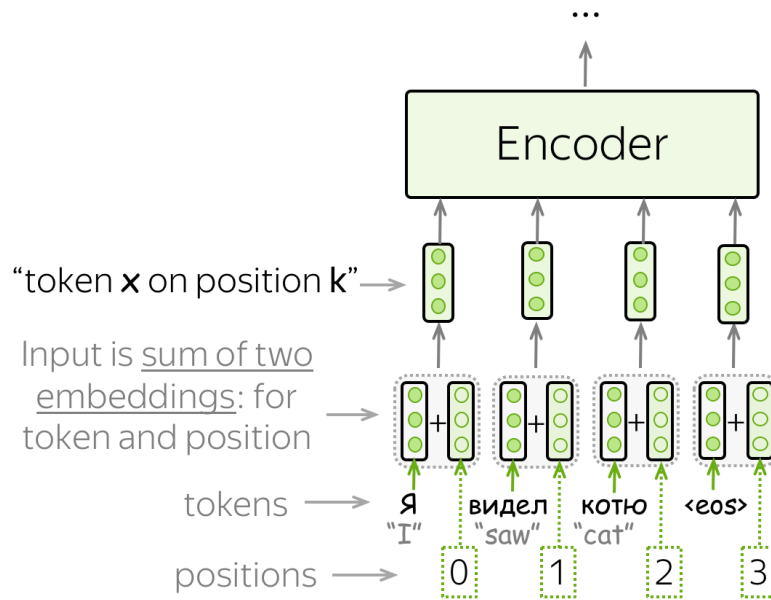
$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

$$PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}})$$

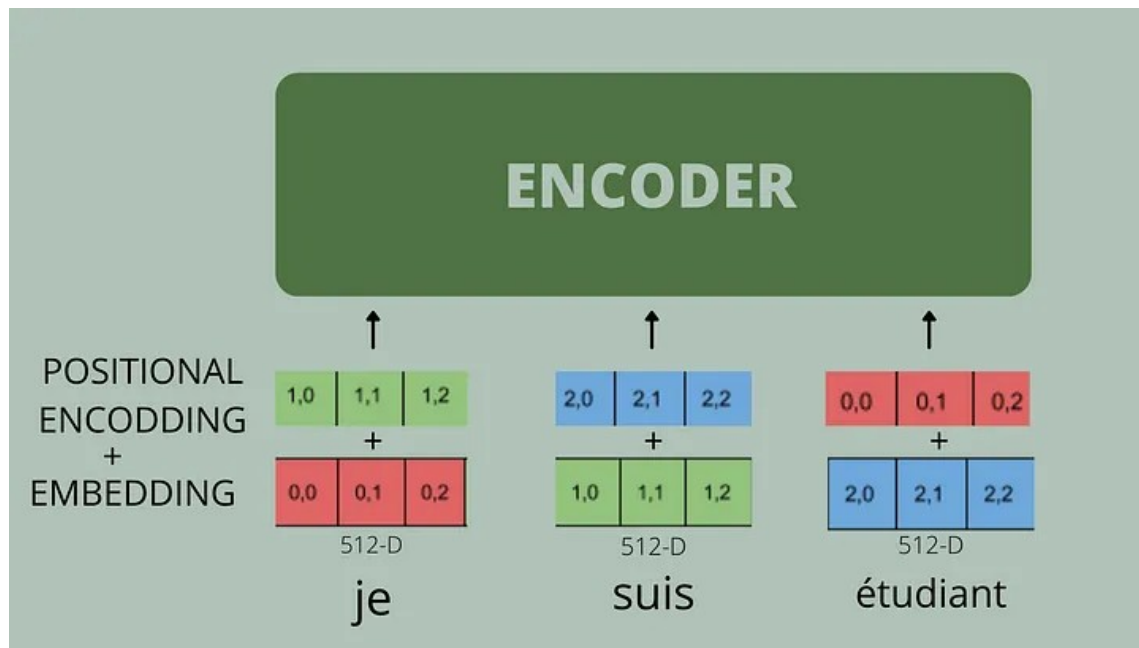
pos : posição

i : dimensão do vetor

d_{model} : dimensão do modelo



Codificação Posicional

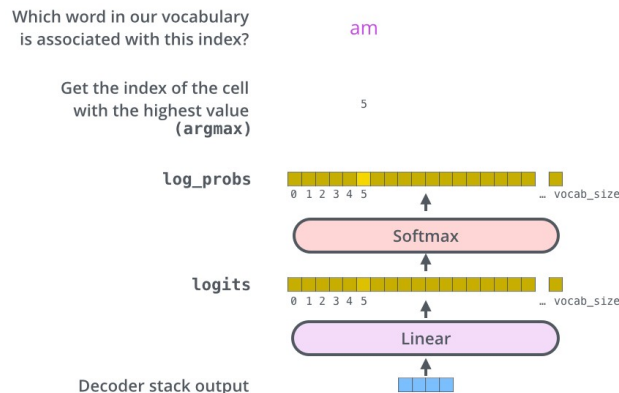


Camadas Linear Final e Softmax

A saída da pilha do *Decoder* retorna um vetor de *floats*. Como torná-lo uma palavra?

A camada linear é uma rede densa contendo n células, onde n é o tamanho do vocabulário. Cada célula corresponde à pontuação (*score*) de uma palavra distinta. Esse vetor de n elementos é chamado de *logits*.

A camada *Softmax* transforma as pontuações em probabilidades. A palavra associada à célula com mais alta probabilidade é produzida como saída.





Representação de Palavras

Representação de Palavras



Palavras precisam ser convertidas para números para serem usadas em aprendizagem de máquina

Vários métodos de representação:

- Pesquisa em um Dicionário

- One-Hot Encoding

- Representação Distribucional

Pesquisa em um Dicionário



Cada palavra distinta do vocabulário recebe um identificador único

Palavra	ID
computador	0
mouse	1
teclado	2
...	...
“fora do dicionário”	n

Pesquisa em um Dicionário



Cada palavra distinta do vocabulário recebe um identificador único

Problema:

O modelo pode incorretamente assumir a existência de uma ordem natural, dando maior importância para as palavras com maior ID

Palavra	ID
computador	0
mouse	1
teclado	2
...	...
“fora do dicionário”	n

One-Hot Encoding



Cada palavra é representada como um vetor de dimensão $N+1$, onde N é o tamanho do dicionário

Para cada palavra, somente a dimensão correspondente à palavra é preenchida com valor 1, e as restantes, com valor zero

Problema: vetor esperso, requerendo muita memória

One-Hot Encoding



	computador	mouse	teclado	“desconhecido”
computador	1	0	0	0
mouse	0	1	0	0
teclado	0	0	1	0
blablablá	0	0	0	1

Representação Distribucional



Cada palavra é representada por um vetor de dimensão fixa N (chamado ***word embedding***), onde cada dimensão representa um contexto.

Intuição: palavras que têm o mesmo significado têm uma representação similar.

Representação Distribucional



	gênero	realiza	idade	pet
homem	-0,98	0,02	0,01	0,02
mulher	0,99	0,01	0,02	0,02
rei	-0,96	0,95	0,71	0,03
rainha	0,98	0,94	0,69	0,04
cachorro	-0,99	-0,01	0,01	0,99
gato	-0,99	0,00	-0,01	0,95

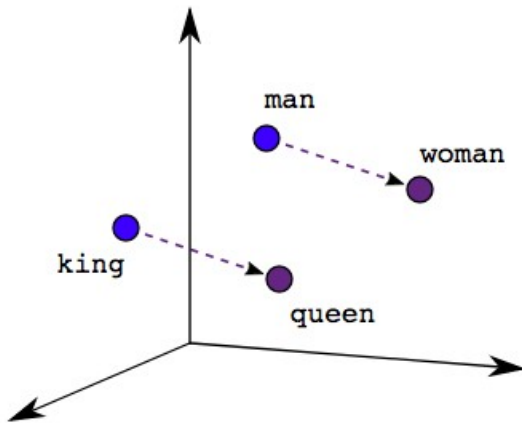
Representação Distribucional



	gênero	realiza	idade	pet
homem	-0,98	0,02	0,01	0,02
mulher	0,99	0,01	0,02	0,02
rei	-0,96	0,95	0,71	0,03
rainha	0,98	0,94	0,69	0,04
cachorro	-0,99	-0,01	0,01	0,99
gato	-0,99	0,00	-0,01	0,95



Representação Distribucional



	gênero	realeza	idade	pet
man	-0,98	0,02	0,01	0,02
woman	0,99	0,01	0,02	0,02
king	-0,96	0,95	0,71	0,03
queen	0,98	0,94	0,69	0,04

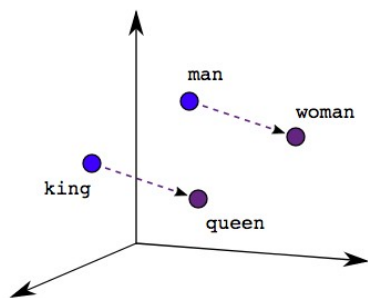
$$\text{man} - \text{woman} = [-1,97, 0,01, -0,01, 0,00]$$

$$\text{king} - \text{queen} = [-1,94, 0,01, 0,02, -0,01]$$

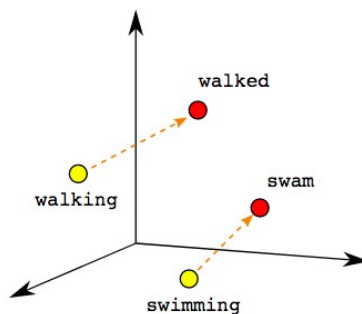
$$\text{man} - \text{woman} \approx \text{king} - ?$$

$$\text{queen} \approx \text{king} - \text{man} + \text{woman}$$

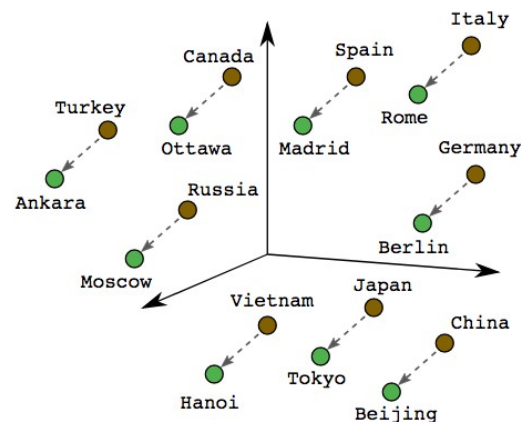
Representação Distribucional



Male-Female



Verb Tense



Country-Capital

Representação Distribucional

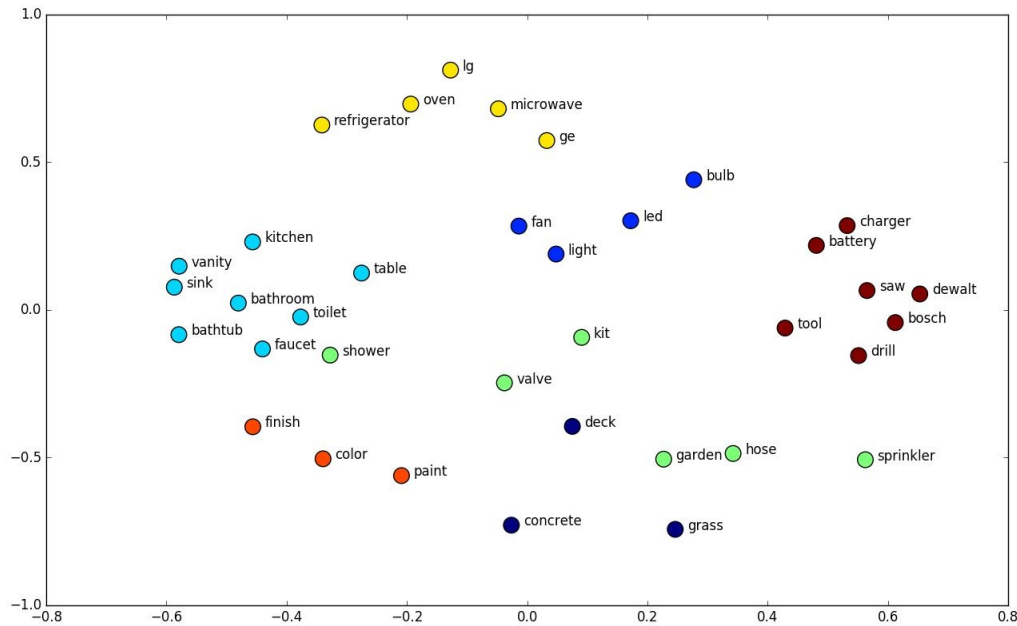
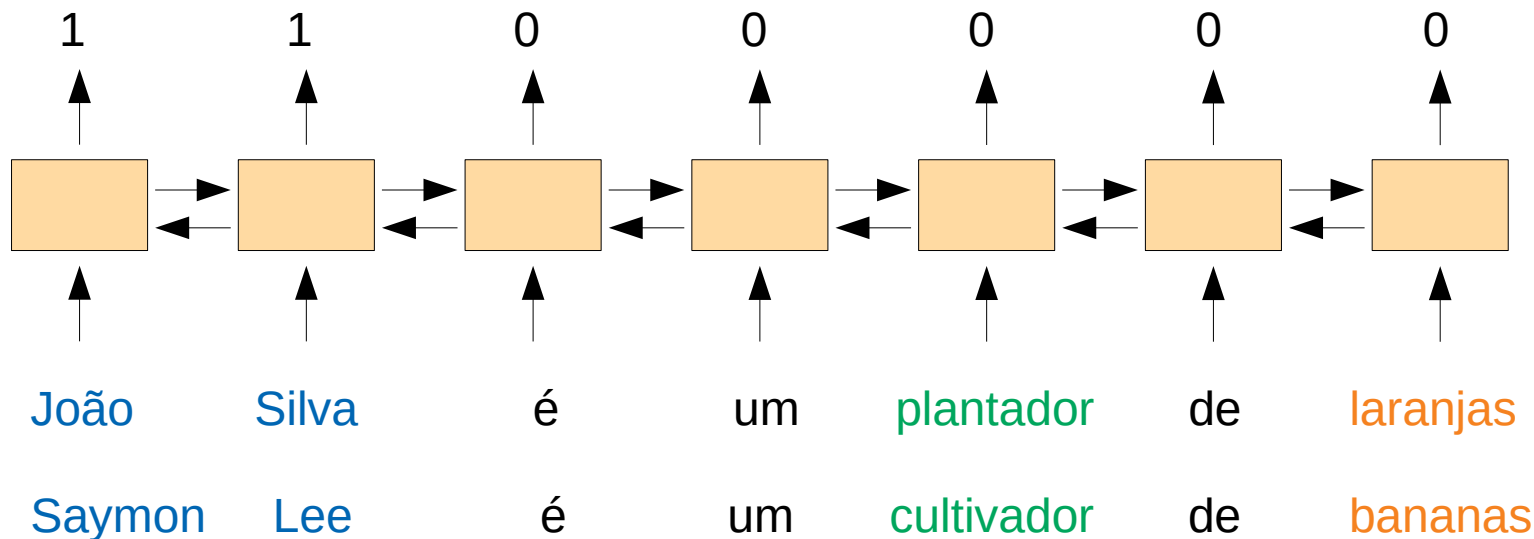


Gráfico gerado pelo
método t-SNE
(t-distributed stochastic
neighbor embedding)

Usando os *Word Embeddings*

Exemplo: Reconhecimento de Entidades Nomeadas



Representação Distribucional



Principais implementações para geração de *Word Embeddings*:

Word2Vec

Glove

FastText

Elmo

BERT

Modelos de *Word Embeddings*



Os modelos podem ser usados das seguintes formas:

- Podem ser treinados usando o seu próprio dataset

- Podem ser usados modelos pré-treinados disponíveis publicamente

Transfer Learning: use um *embedding* pré-treinado e faça o ajuste fino (*fine tuning*) para uma tarefa específica por meio de treinamento

Word2Vec



Principais abordagens

CBOW – *Continuous bag-of-word*

determina a palavra dado o contexto

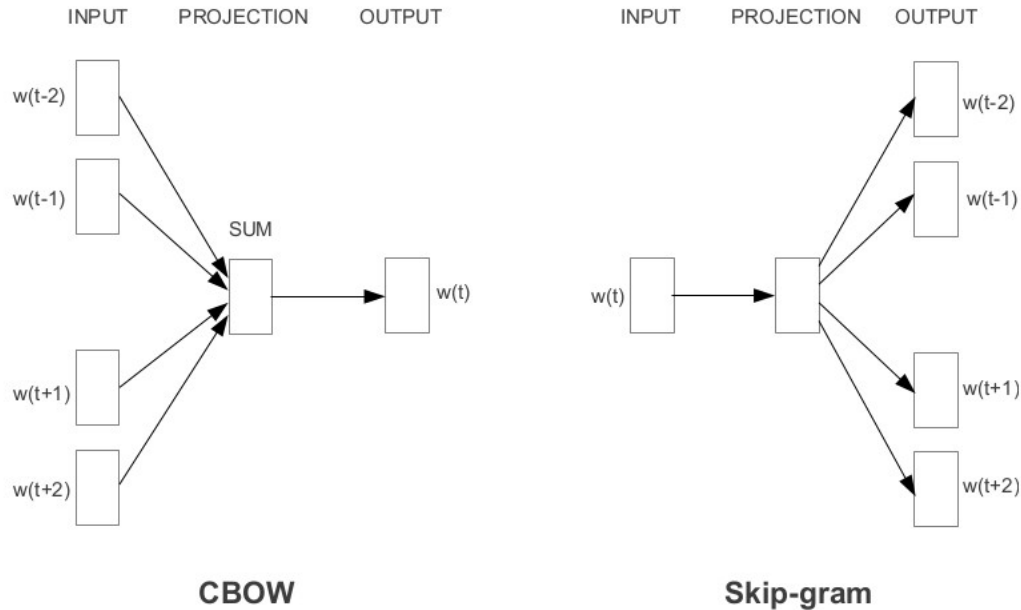
a ordem das palavras no contexto não influencia
a predição (*bag-of-word*)

Skip-gram

determina o contexto dada uma palavra

Mikolov et al. (2013). Efficient Estimation of Word Representations in Vector Space

Modelos CBOW e Skip-gram



Mikolov et al. (2013). Efficient Estimation of Word Representations in Vector Space

GloVe (*Global Vectors*)



Explora estatísticas de co-ocorrências de palavras globalmente em um corpus (Word2Vec captura contexto local)

Constrói uma matriz de co-ocorrências

Computa $P(w_1|w_2)$

Reduz a alta dimensão da matriz por normalização

Pennington, et al. (2014). GloVe: Global Vectors for Word Representation

Problemas com Word2Vec e GloVe



Não generalizam para palavras fora do vocabulário

Palavras que não estavam no conjunto de treino

Palavras raras não são bem aprendidas

Palavras com múltiplos significados (polissemia) têm a mesma representação

Maria foi ao **banco** pedir um empréstimo

Maria sentou no **banco** da praça

=> *word embeddings* estáticos

FastText



Extensão do Skip-gram (Word2Vec)

Quebra as palavras em vários n-gramas (*sub-words*)

O vetor *word embedding* de uma palavra é dado pela soma de seus n-gramas

Os n-gramas de palavras fora do vocabulário provavelmente aparecem em outras palavras do conjunto de treino



Modelo de Linguagem

Modelo de Linguagem



Determina a próxima palavra com base em um modelo probabilístico

Qual a probabilidade de cada palavra do vocabulário ser a próxima palavra da frase:

Um especialista na produção de vinhos é um _____

Modelo treinado em um grande corpus linguístico

Modelo de Linguagem



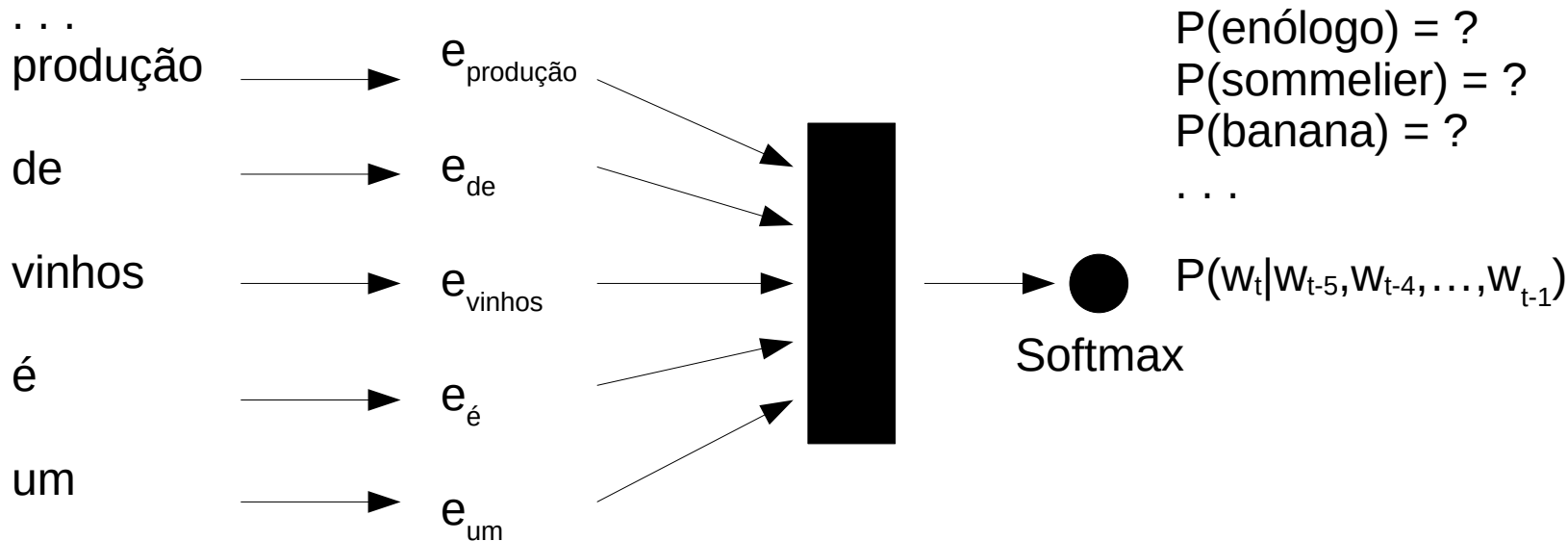
Um especialista na produção de vinhos é um enólogo

$P(\text{Um especialista na produção de vinhos é um enólogo}) = P(\text{Um}) \cdot$
 $P(\text{especialista} \mid \text{Um}) \cdot P(\text{na} \mid \text{Um especialista}) \cdot P(\text{produção} \mid \text{Um especialista na}) \cdot P(\text{de} \mid \text{Um especialista na produção}) \cdot \dots$

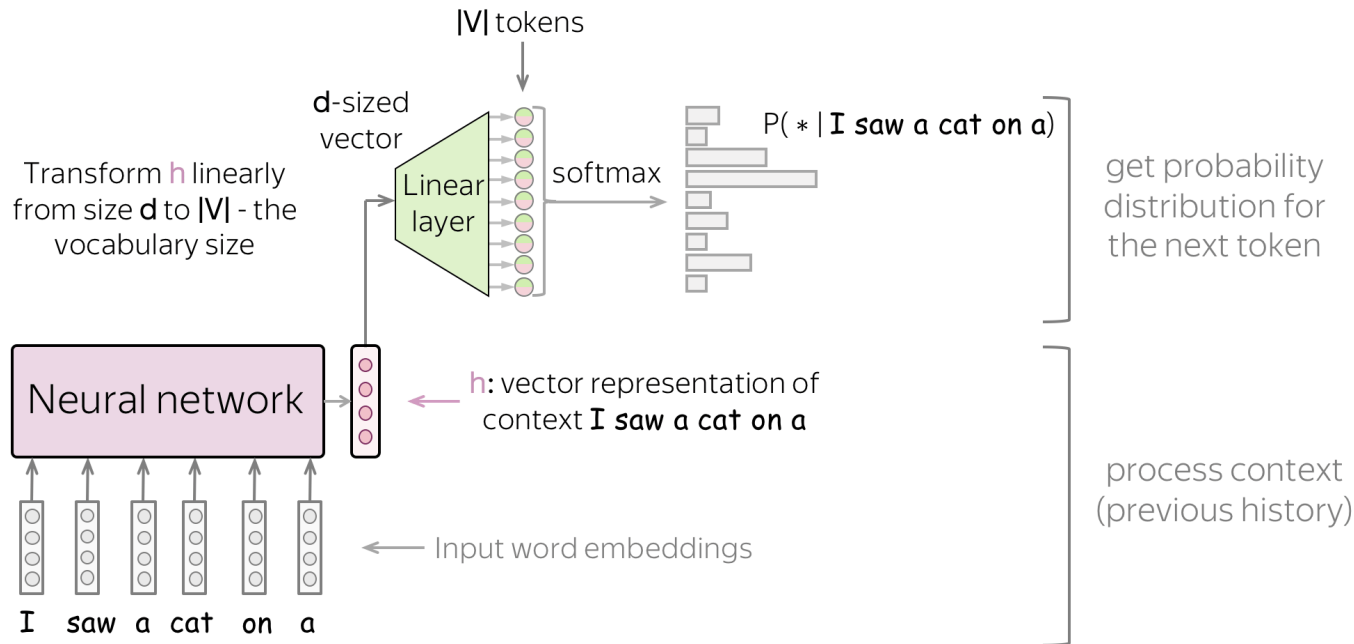
$$P(y_1, y_2, \dots, y_n) = P(y_1) \cdot P(y_2|y_1) \cdot P(y_3|y_1, y_2) \cdot \dots \cdot P(y_n|y_1, \dots, y_{n-1}) = \prod_{t=1}^n P(y_t|y_{<t})$$

Modelo de Linguagem

Um especialista na produção de vinhos é um enólogo

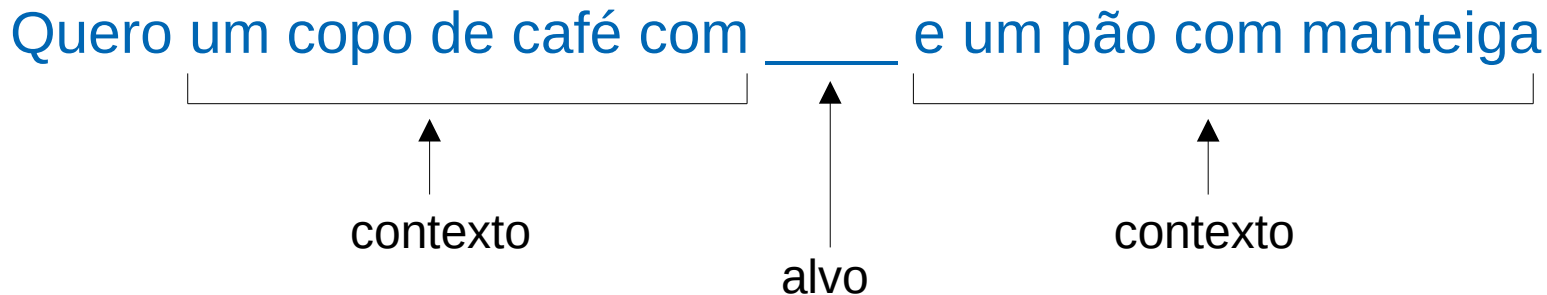


Modelo de Linguagem



Fonte: https://lena-voita.github.io/nlp_course/language_modeling.html

Modelo de Linguagem



Contexto:

- 5 palavras antes e 5 palavras depois
- últimas 4 palavras
- últimas 2 palavras

$P(\text{leite} \mid \text{contexto}) = ?$
 $P(\text{chá} \mid \text{contexto}) = ?$
 $P(\text{nescau} \mid \text{contexto}) = ?$

ELMo (*Embeddings from Language Models*)



Produz *word embeddings* dinamicamente, levando em consideração o contexto da sequência de palavras de entrada

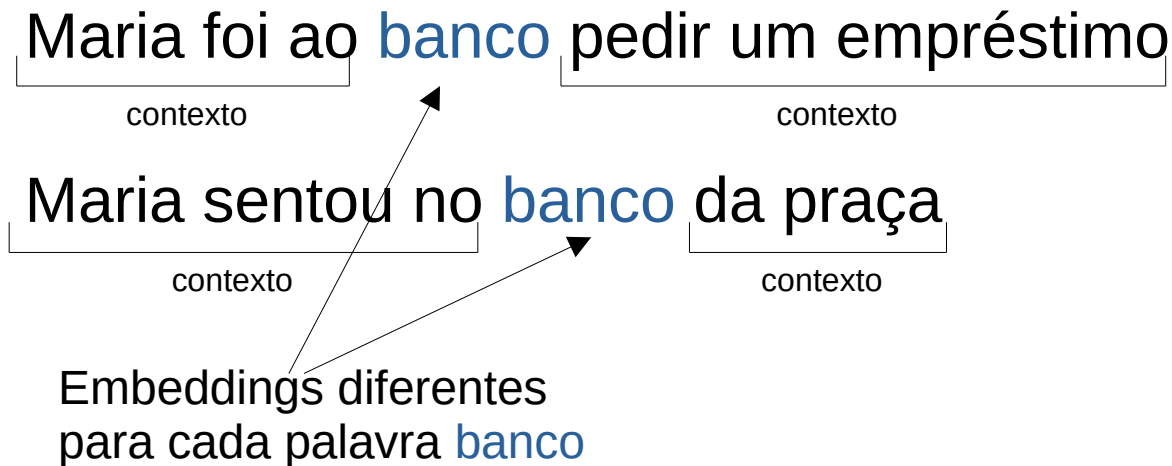
É um modelo de linguagem

Treinado para prever a próxima palavra de uma sequência

Word2Vec, GloVe e FastText produzem simples *word embeddings* estáticos (não são modelos de linguagem)

ELMo (*Embeddings from Language Models*)

Considera o contexto anterior e posterior à palavra alvo



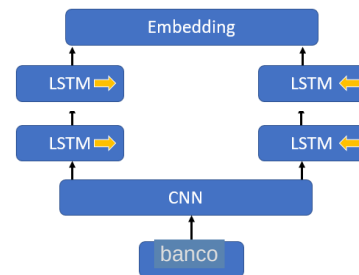
ELMo (*Embeddings from Language Models*)

Arquitetura consistindo de uma rede neural recorrente LSTM bidirecional

Modelo de entrada puramente baseado em caractere

Alimentado em uma rede convolucional (CNN)

Permite representação robusta para palavras fora do vocabulário



ELMo versus FastText



FastText é muito rápido e incorpora n-gramas eficientemente

Porém, produz *word embeddings* estáticos

ELMo considera o contexto para produzir os *word embeddings* (*word embeddings* dinâmicos)

Qualidade superior aos modelos anteriores

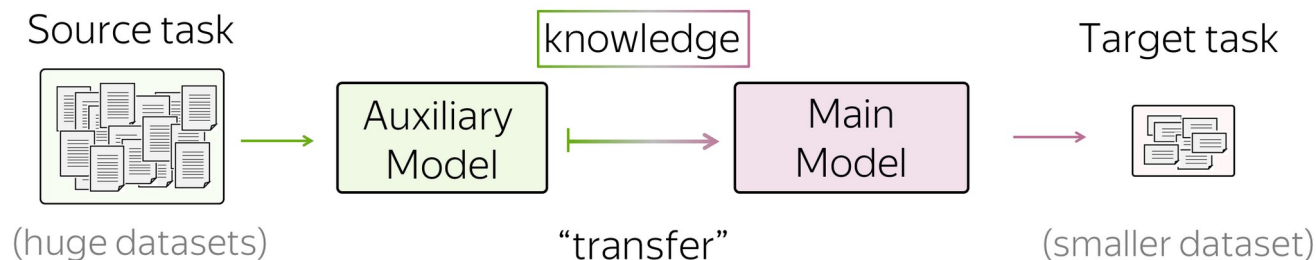
Tem um custo maior por fazer previsões em tempo de execução



Transferência de Aprendizagem

Transferência de Aprendizagem

Transferência de Aprendizagem (*Transfer Learning*) significa transferir conhecimento de um modelo para outro



Referência: https://lena-voita.github.io/nlp_course/transfer_learning.html

Transferência de Aprendizagem

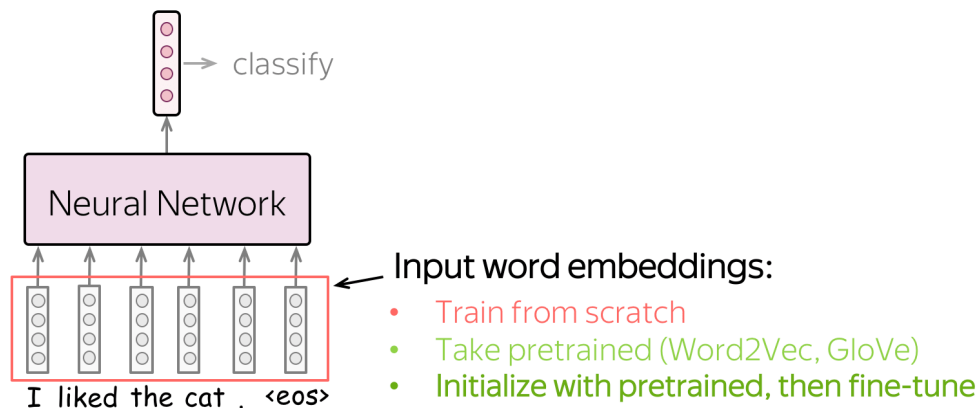


Use um modelo pré-treinado e faça o ajuste fino (*fine tuning*) para uma tarefa específica por meio de treinamento

No *fine tuning*, você parte de um modelo pré-treinado em uma grande quantidade de dados, em uma tarefa que você não está interessado, e o treina (ajusta) para a tarefa que você está interessado (ex: classificação de sentimentos) usando uma taxa de aprendizagem bem pequena

Transferência de Aprendizagem

Word Embeddings são uma forma simples de transferência de aprendizagem



Referência: https://lena-voita.github.io/nlp_course/transfer_learning.html

Transferência de Aprendizagem

- Train from scratch

What they will know:

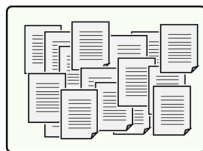


May be not enough to learn relationships between words

Modelo conhece apenas os dados de classificação

- Take pretrained (Word2Vec, GloVe)

What they will know:

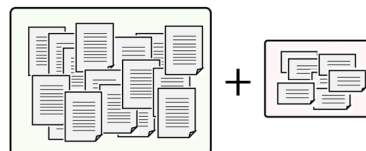


Know relationships between words, but are **not** specific to the task

Parâmetros fixos
Vetores estáticos

- Initialize with pretrained, then fine-tune

What they will know:



Know relationships between words and adapted for the task

Parâmetros pré-treinados são ajustados por uma pequena taxa de aprendizagem

“Transfer” knowledge from a huge unlabeled corpus to your task-specific model

Referência: https://lena-voita.github.io/nlp_course/transfer_learning.html

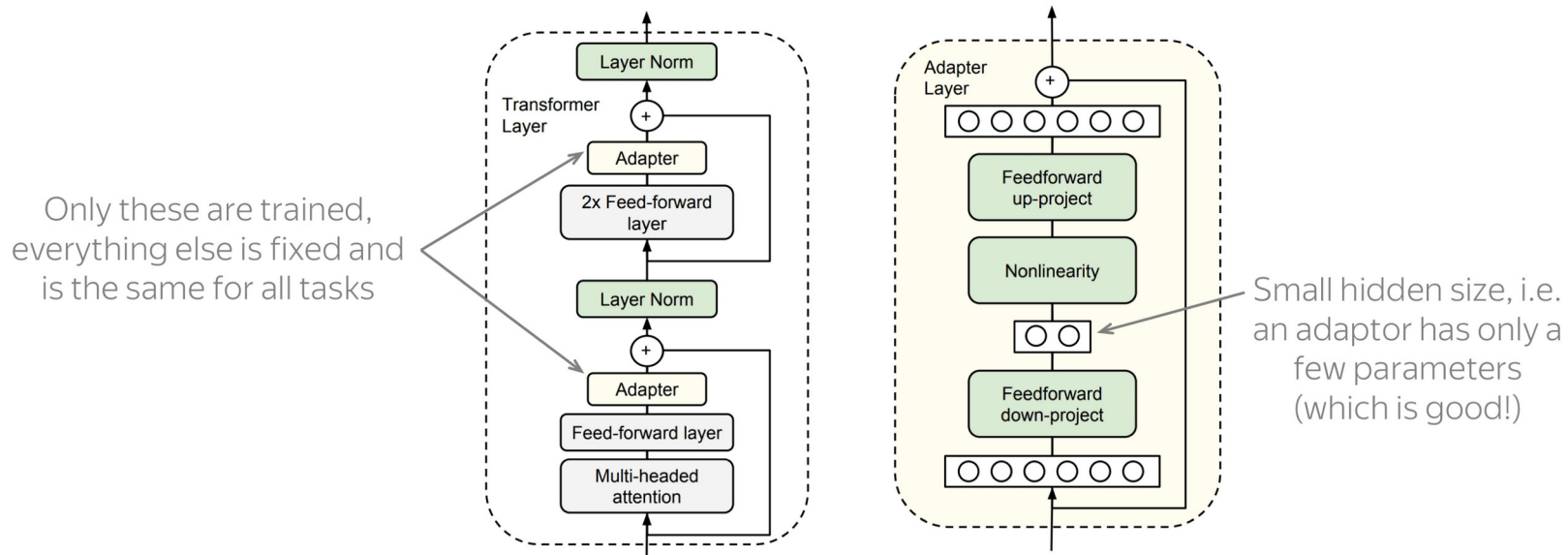
Adaptadores



O processo de *fine tuning* implica em atualizar todo o grande modelo para cada tarefa, o que é muito ineficiente

Adaptadores são módulos adicionados à arquitetura do modelo contendo uns poucos parâmetros atualizáveis para cada tarefa, enquanto os parâmetros do modelo original ficam fixos. Isso torna o treinamento muito mais rápido

Exemplo de um Módulo Adaptador



Referência: Housby et al. Parameter-Efficient Transfer Learning for NLP <https://arxiv.org/pdf/1902.00751.pdf>



Modelos de Linguagem com *Fine Tuning*

Modelos de Linguagem com *Fine Tuning*



Os modelos mais antigos são usados para substituir a camada de *embedding*, e as arquiteturas desses modelos são específicas para cada tarefa

Uma arquitetura diferente para cada tarefa

Nos modelos a seguir, a mesma arquitetura é usada para todas as tarefas, sendo o modelo ajustado para cada uma

Para cada tarefa, o que muda é a forma de alimentar os dados

BERT (*Bidirectional Encoder Representations from Transformers*)



Produz *word embeddings* dinamicamente, levando em consideração o contexto da sequência de palavras de entrada, da mesma forma que o ELMo

É um modelo de linguagem

Pré-treinado em duas diferentes tarefas:

Predição da palavra mascarada

Predição da próxima sentença

Paradigma “*pre-training, fine-tuning*”

BERT - *Pre-training, Fine-tuning*



Pré-treino

Treinamento auto-supervisionado (*self-supervised*) em um grande corpus de texto

Ajuste fino (*Fine-Tuning*)

Treinamento supervisionado com uma quantidade limitada de dados para uma tarefa específica

BERT - *Pre-training*



Predição da palavra mascarada (MLM)

Sentença:

o rato roeu a roupa do rei

Sentença mascarada:

o rato [*mask*] a roupa do rei

Treinado para prever a palavra mascarada

15% das palavras são mascaradas aleatoriamente

BERT - *Pre-training*



Predição da próxima sentença (NSP)

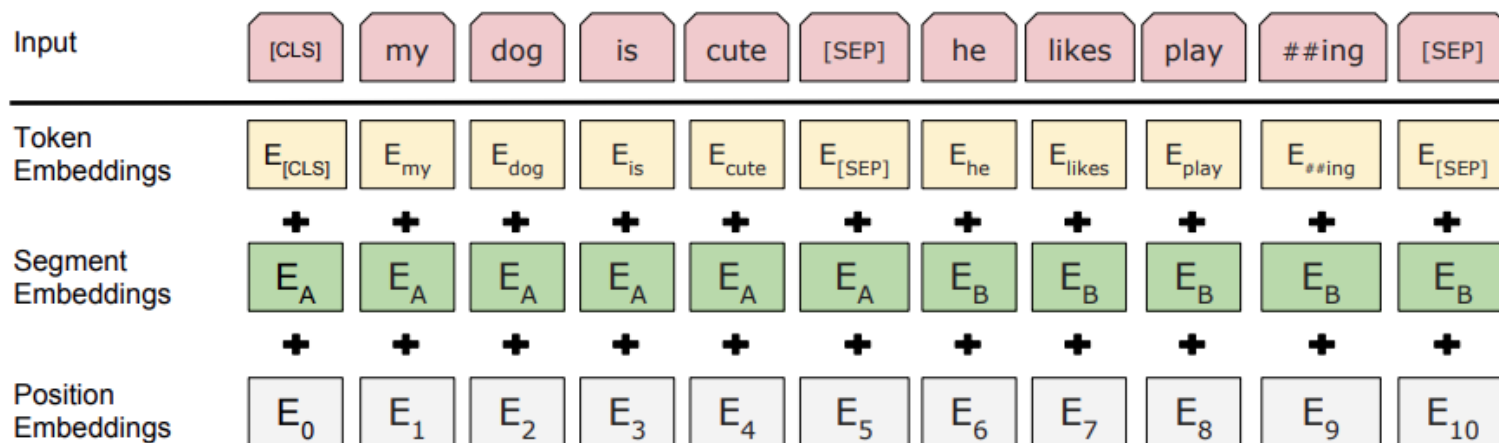
Sentença 1: João foi ao posto de gasolina

Sentença 2: Ele encheu o tanque do carro

Treinado para predizer se as duas sentenças são sentenças consecutivas ou não em um texto

Treinado com sentenças consecutivas (50%) e não consecutivas (50%, geradas aleatoriamente)

BERT - *Pre-training*



Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

BERT - *Pre-training*



[CLS] é um token de classificação binária, adicionado no início da primeira sentença para predizer se a segunda sentença segue a primeira

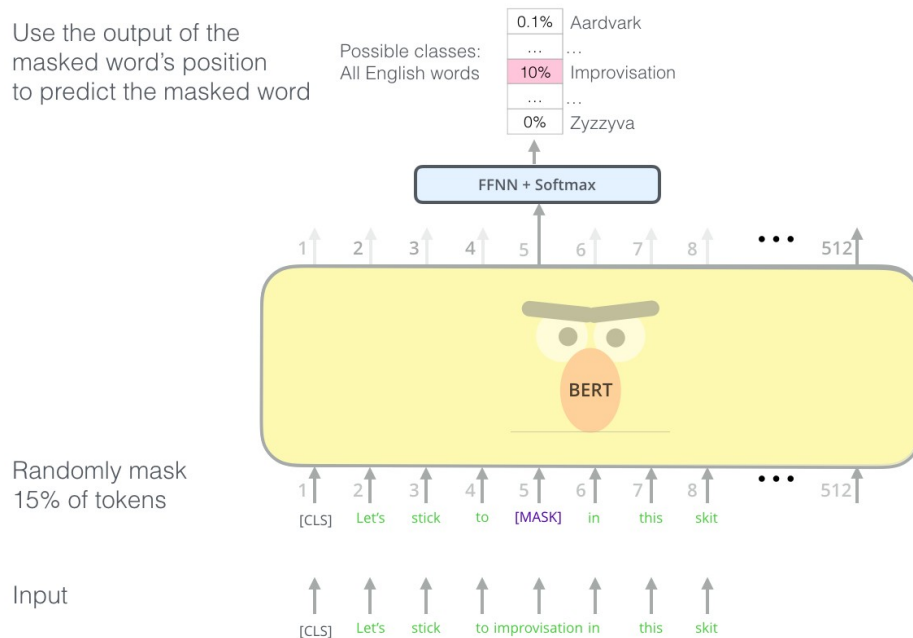
[SEP] é um token de separação que sinaliza o fim de uma sequência

##xxx indica tokens subsequentes de uma palavra

Uma palavra pode ser quebrada em várias partes (*subwords*)

BERT - Pre-training

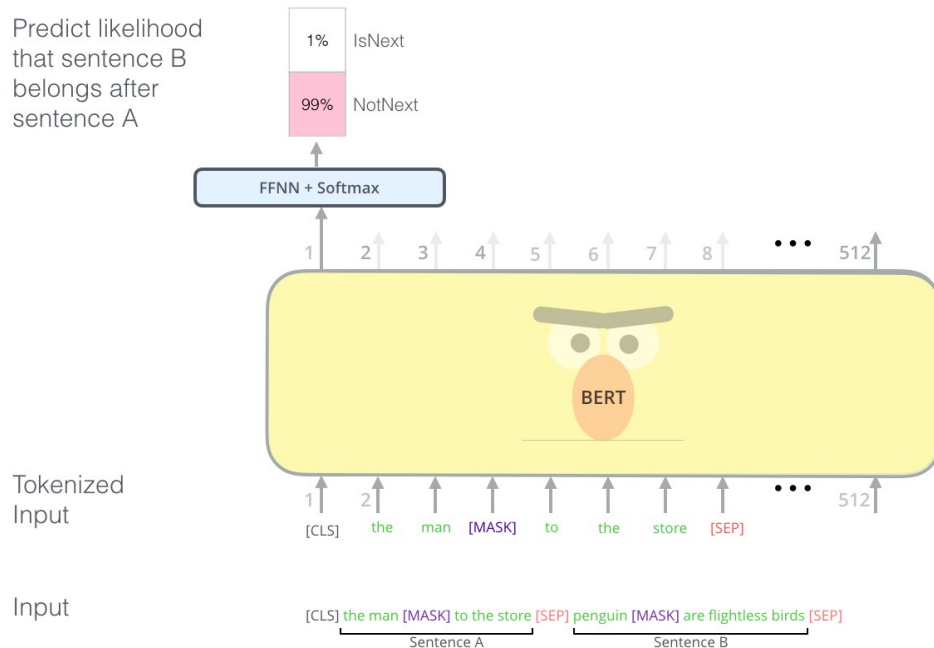
Use the output of the masked word's position to predict the masked word



Alammar (2021). The Illustrated BERT, ELMo, and co. (<http://jalammar.github.io/illustrated-bert/>)

BERT - *Pre-training*

Predict likelihood
that sentence B
belongs after
sentence A



Alammar (2021). The Illustrated BERT, ELMo, and co. (<http://jalammar.github.io/illustrated-bert/>)

BERT - *Fine-tuning*



Carregue uma versão pré-treinada do modelo BERT

Configuração das camadas da rede e seus pesos

Treine o modelo para uma tarefa específica

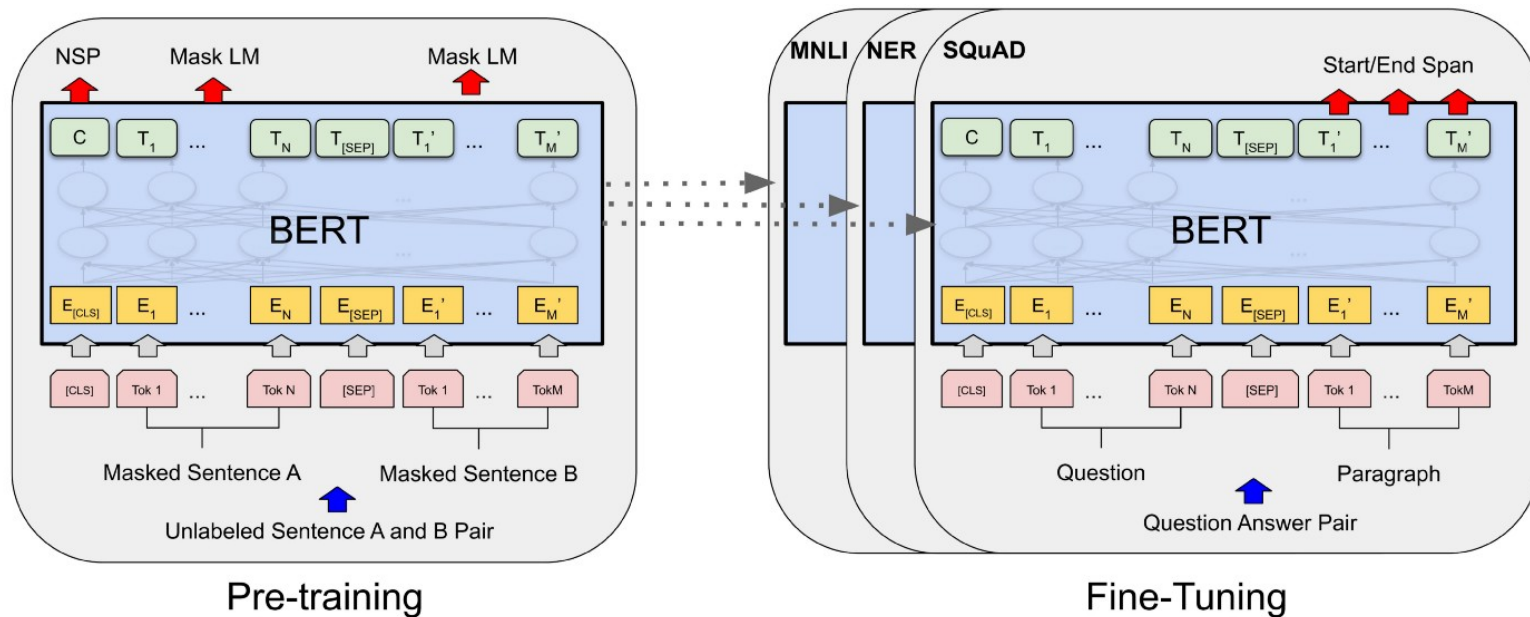
Question-Answer, NER, Análise de sentimentos,

Tradução, Sumarização, ...

Treina os pesos das camadas do BERT e do
classificador conjuntamente

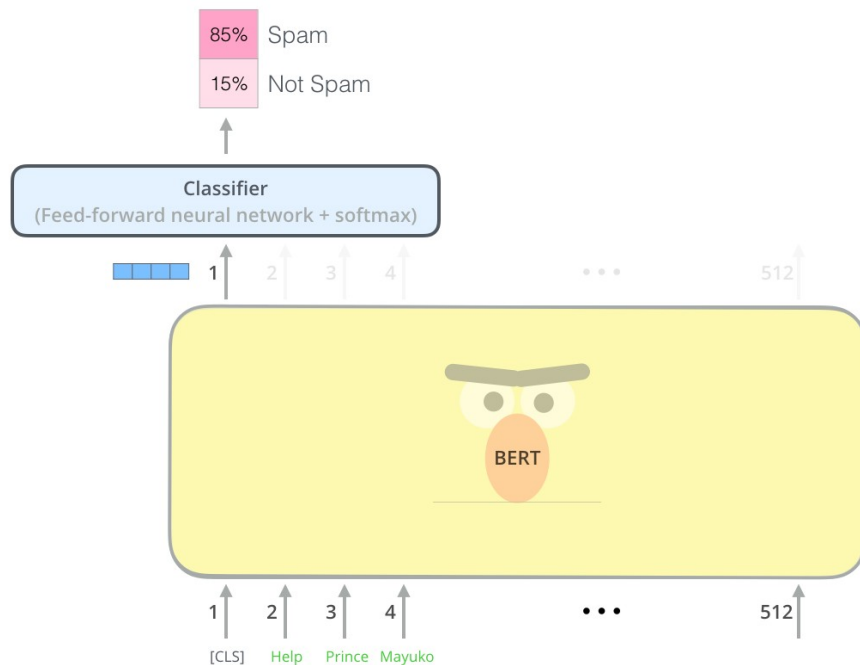
Camada de saída adicional

BERT - *Pre-training, Fine-tuning*



Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

BERT - *Fine-tuning*: Classificação

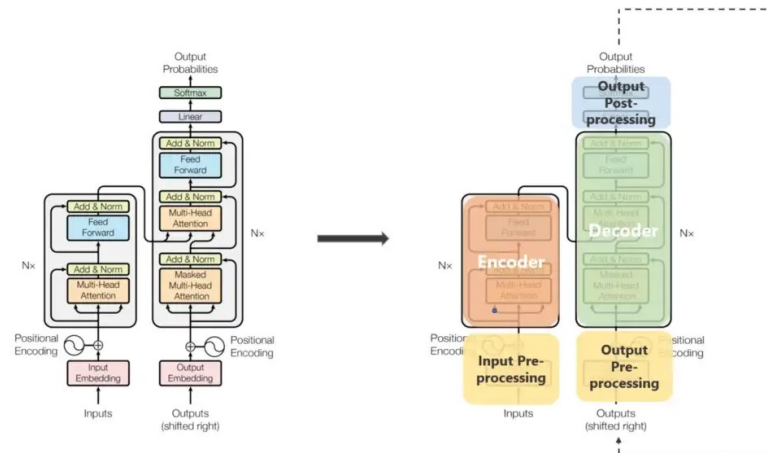


Alammar (2021). The Illustrated BERT, ELMo, and co. (<http://jalammar.github.io/illustrated-bert/>)

BERT - Arquitetura

BERT usa um *Encoder Transformer* multi-camada bidirecional

A camada de *self-attention* executa *self-attention* em ambas as direções



Vaswani et al. (2017). Attention Is All You Need

BERT - Modelos Pré-treinados



Google liberou duas variantes do modelo:

BERT Base: 12 camadas, 110M de parâmetros

BERT Large: 24 camadas, 340M de parâmetros

Treinado em toda a Wikipedia e no Book Corpus

Vários modelos disponíveis em:

Tensorflow Hub

Hugging Face Hub

BERT - Variantes



DistilBERT

versão menor e mais rápida do BERT

preserva 95% de sua performance

ALBERT

versão mais leve e mais rápida, com redução de parâmetros para diminuir o consumo de memória

Substituição da Predição da Próxima Sentença por Predição da Ordem das Sentenças

BERT - Variantes



RoBERTa

Masked Language Modeling dinâmica

Não usa a Predição da Próxima Sentença

Usa o tokenizador BPE

Melhora os hiperparâmetros

XLNet-RoBERTa

RoBERTa treinado em um corpus multi-linguagem com

Masked Language Modeling

GPT (*Generative Pre-trained Transformer*)



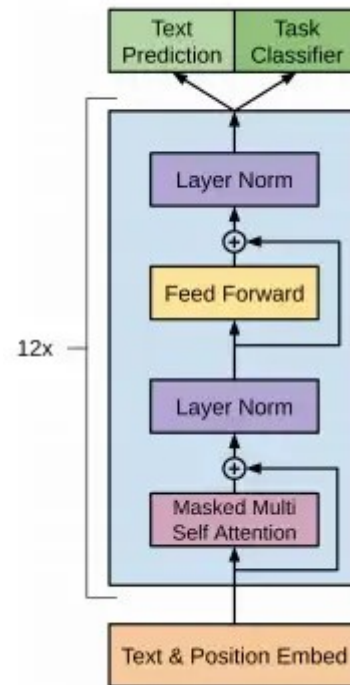
GPT-1, GPT-2, GPT-3 e GPT-4 são modelos de linguagem introduzidos pela OpenAI

Objetivo: generalizar o conceito de compreensão da linguagem para qualquer tarefa específica

GPT - Arquitetura

GPT usa somente o *Decoder* da arquitetura do *Transformer* original, e é unidirecional

GPT-4 possui 1,76 trilhão de parâmetros



GPT - Ajuste para Tarefa Específica



Fine-tuning: muitos exemplos de treino para atualização dos pesos

Few-shot: poucos exemplos de treino para condicionamento, sem atualização de pesos

One-shot: um exemplo de treino, sem atualização de pesos

Zero-shot: nenhum exemplo de treino, sem atualização de pesos

GPT - Ajuste para Tarefa Específica

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



GPT - Ajuste para Tarefa Específica

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



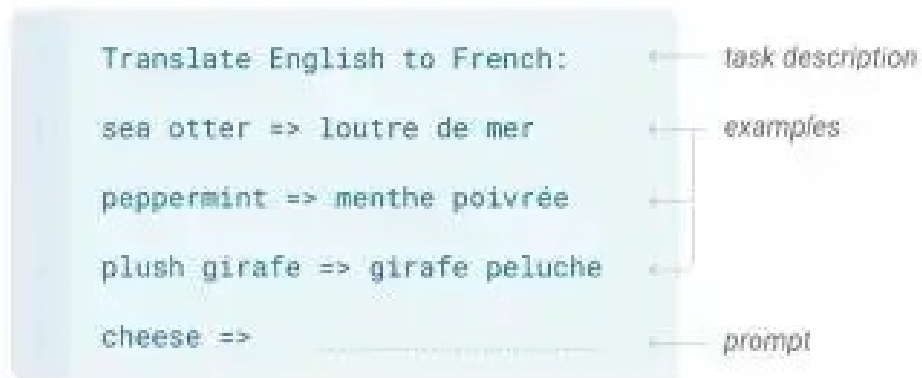
The diagram illustrates a one-shot prompt structure. It consists of three lines of text, each with a label to its right connected by a horizontal line. The first line is 'Translate English to French:' labeled 'task description'. The second line is 'sea otter => loutre de mer' labeled 'example'. The third line is 'cheese =>' labeled 'prompt'. The text is displayed in a light blue monospace font on a white background.

```
Translate English to French:      task description
sea otter => loutre de mer       example
cheese =>                        prompt
```

GPT - Ajuste para Tarefa Específica

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



Brown et al. (2020). Language Models are Few-Shot Learners

GPT - Ajuste para Tarefa Específica

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



Brown et al. (2020). Language Models are Few-Shot Learners

Cadeia de Pensamento

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

<https://medium.com/@haifengl/a-tutorial-to-llm-f78dd4e82efc>

Chatbot



Objetivo de um Modelo de Linguagem:

Predizer a próxima palavra

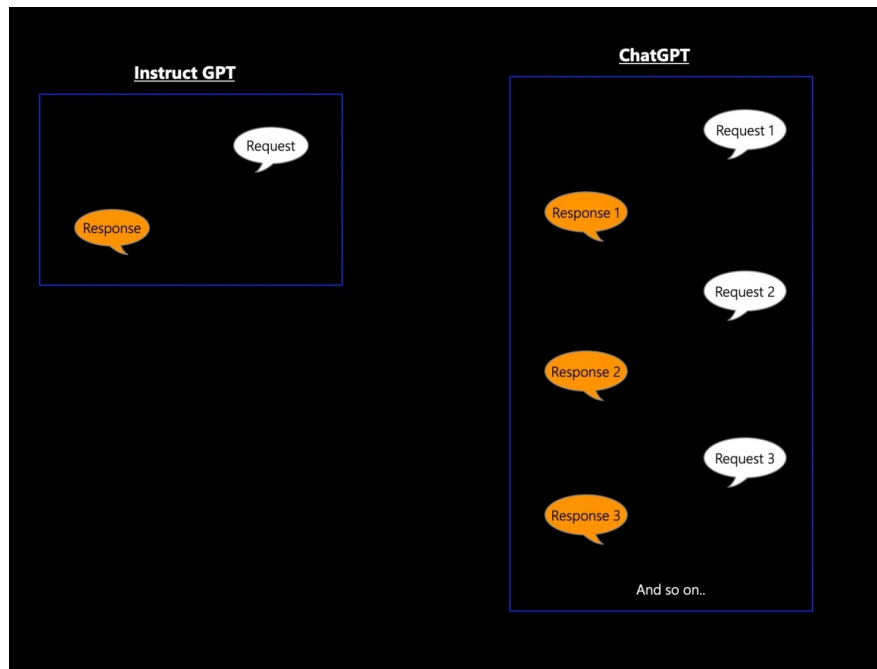
Objetivo de um *chatbot*:

Seguir as instruções do usuário para produzir conteúdo útil e seguro, mantendo a conversa entre eles

InstructGPT e ChatGPT

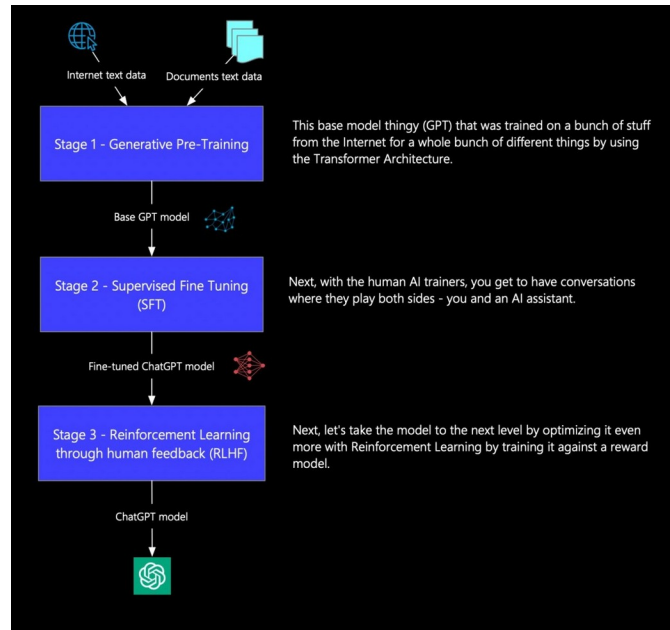
InstructGPT é o precursor do ChatGPT

InstructGPT alinha um modelo de linguagem com a intenção do usuário em uma ampla gama de tarefas por meio da aprendizagem por reforço com retorno do usuário



Ouyang et al. (2022). Training language models to follow instructions with human feedback

Estágios de Treinamento do ChatGPT



Estágio 1: Treina um Transformer como um modelo de linguagem (Modelo GPT)

Estágio 2: Executa um *Fine-tuning* supervisionado em tarefas específicas de conversação

Estágio 3: Executa um treinamento por reforço, onde especialistas humanos ranqueiam as respostas do *chatbot* em ordem de relevância

<https://www.linkedin.com/pulse/discover-how-chatgpt-istrained-pradeep-menon>

Outros Modelos de Linguagem

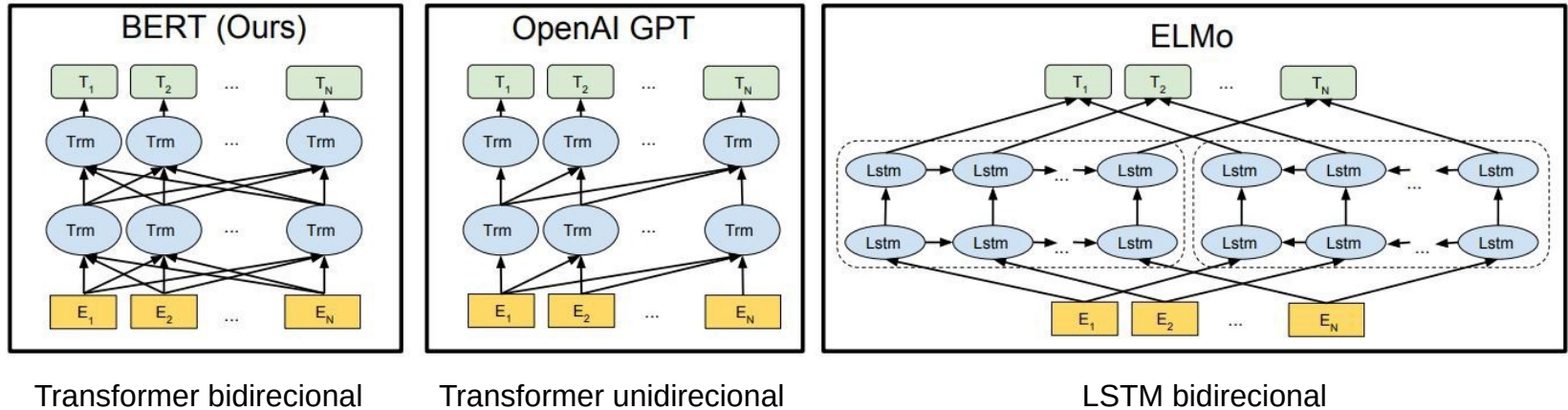


Diversos modelos têm surgido

CTRL, XLM, XLNet, BART, T5, ELECTRA, Switch
Transformers, BLOOM, Gemini, Llama, Mistral, Sabiá

<https://medium.com/nlplanet/a-brief-timeline-of-nlp-from-bag-of-words-to-the-transformer-family-7caad8bbba56>

Comparação de Diferentes Modelos



Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Comparação de Diferentes Modelos

Encoder Model	text classification named entity recognition sentiment analysis extractive question answering	BERT DistilBERT RoBERTa ALBERT
Decoder Model	casual language modeling generating sequences	GPT families CTRL Transformer XL
Sequence-to-sequence Model	text summarization translation generative question answering	Pegasus T5 FLAN-T5 BART mBART

Referência: <https://alina-li-zhang.medium.com/chatgpt-and-other-transformers-how-to-select-large-language-model-for-your-nlp-projects-908de1a152d8>



Representação de Documentos

Representação de Documentos



Documentos são compostos por palavras

Algoritmos de Aprendizagem de Máquina processam valores numéricos

Os textos são convertidos em uma representação numérica por meio de *embeddings*

Vetores numéricos n-dimensionais

Representação de Documentos



Objetivo: representar todo o documento por meio de um único *embedding*

Principais formas de representação:

Bag of words (Saco de Palavras)

Esquema de pesos TF-IDF

Modelos baseados em *deep learning*

SentenceBERT

Representação de Documentos



Principais aplicações

Busca semântica

Dada uma consulta, encontrar os documentos mais similares a ela (máquina de busca)

Comparar a similaridade semântica entre dois documentos

Classificação de textos

Classificar documentos em categorias pré-definidas



Aplicações PLN

Principais Aplicações



Busca

Rankeia documentos em ordem de relevância para uma consulta

Classificação de textos

Classifica documentos por similaridade em relação à categorias pré-definidas

Agrupamento (*Clustering*)

Agrupa documentos por similaridade (sem categorias pré-definidas)

Principais Aplicações



Classificação de *Tokens*

Atribui um rótulo de classe a cada *token* (palavra) em uma sentença

Reconhecimento de Entidades Nomeadas
(NER – *Named Entity Recognition*)

Classificação Gramatical de Palavras
(*Part-of-Speech Tagging*)

Principais Aplicações



Resposta a Pergunta (*Question Answering*)

Retorna uma resposta dada uma pergunta

Assistentes virtuais como Alexa, Siri ou Google

Dois tipos de tarefas:

Extrativa: extrai a resposta de um dado contexto

Abstrativa: gera uma resposta a partir do contexto

Principais Aplicações



Tradução

Converte uma sequência de texto de uma linguagem para outra

Tradução de idiomas

Conversão de texto para fala ou fala para texto

Principais Aplicações



Sumarização

Cria uma versão mais curta de um documento capturando todas as informações importantes

Dois tipos de tarefas:

Extrativa: extrai a informação relevante

Abstrativa: gera um novo texto com a informação relevante

Principais Aplicações



Recomendação

Recomenda itens relacionados a um determinado texto

Recomendação de filmes, músicas e produtos

Chatbot

Simula conversação com seres humanos



Referências Bibliográficas

Referências Bibliográficas



1. Sandro Skansi. Introduction to Deep Learning: From Logical Calculus to Artificial Intelligence. Springer, 2018.

<https://doi.org/10.1007/978-3-319-73004-2>

2. Data Science Academy. Deep Learning Book.

<http://deeplearningbook.com.br/>

3. Ilya Sutskever, Oriol Vinyals, Quoc V. Le. Sequence to Sequence Learning with Neural Networks. ArXiv, 2014.

<https://arxiv.org/pdf/1409.3215.pdf>

Referências Bibliográficas



4. Cho et al. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. arXiv:1406.1078.
<https://arxiv.org/abs/1406.1078>
5. Vaswani et al. Attention Is All You Need. arXiv:1706.03762.
<https://arxiv.org/abs/1706.03762>

Links Interessantes



- https://lena-voita.github.io/nlp_course.html
- <https://neptune.ai/blog/recurrent-neural-network-guide>
- https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html
- <https://docs.chainer.org/en/v7.8.0/examples/seq2seq.html>
- <https://luv-bansal.medium.com/transformer-attention-is-all-you-need-easily-explained-with-illustrations-d38fdb06d7db>
- <http://jalammar.github.io/illustrated-transformer/>
- <https://towardsdatascience.com/attention-is-all-you-need-discovering-the-transformer-paper-73e5ff5e0634>

Links Interessantes

- 
- <https://levelup.gitconnected.com/understanding-transformers-from-start-to-end-a-step-by-step-math-example-16d4e64e6eb1>
 - <https://towardsdatascience.com/word-representation-in-natural-language-processing-part-i-e4cd54fed3d4>
 - <https://towardsdatascience.com/word-representation-in-natural-language-processing-part-ii-1aee2094e08a>
 - <https://towardsdatascience.com/word-representation-in-natural-language-processing-part-iii-2e69346007f>
 - <https://medium.com/@RobinVetsch/nlp-from-word-embedding-to-transformers-76ae124e6281>

Links Interessantes



- <https://medium.com/nlplanet/a-brief-timeline-of-nlp-from-bag-of-words-to-the-transformer-family-7caad8bbba56>
- <https://medium.com/@ankiit/word2vec-vs-bert-d04ab3ade4c9>
- <https://alina-li-zhang.medium.com/chatgpt-and-other-transformers-how-to-select-large-language-model-for-your-nlp-projects-908de1a152d8>
- https://lena-voita.github.io/nlp_course/transfer_learning.html
- <https://gluebenchmark.com/>

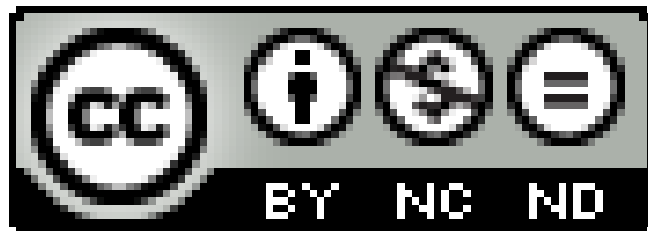


Link da apresentação:

<https://sites.google.com/ufla.br/denilsonpereira/home/courses/deep-learning>

denilsonpereira@ufla.br

<https://sites.google.com/ufla.br/denilsonpereira/>



Este trabalho está licenciado com uma Licença
[Creative Commons - Atribuição-NãoComercial-SemDerivações 4.0](https://creativecommons.org/licenses/by-nc-nd/4.0/)